

# Algoritmi Avansati

Material complet pentru examen  
Seminare + rezolvari + teorie esentiala

**Cum se foloseste materialul.** Prima parte este o fisa de teorie pentru examen: definitii, formule, algoritmi, demonstratii scurte si intrebari probabile. A doua parte contine rezolvarile pas cu pas pentru seminarele lucrate. Pentru Seminarul 4 nu a existat un fisier separat in materialele incarcate; am inclus teoria aferenta din zona Vertex Cover / Programare liniara, care corespunde cursului 4.

# Contents

|                                                             |          |
|-------------------------------------------------------------|----------|
| <b>Surse folosite</b>                                       | <b>2</b> |
| <b>I Fisa de teorie pentru examen</b>                       | <b>3</b> |
| <b>1 Cum gandesti o problema la examen</b>                  | <b>3</b> |
| 1.1 Notatii standard . . . . .                              | 3        |
| 1.2 Complexitati . . . . .                                  | 3        |
| <b>2 P, NP, NP-hard, NP-complet</b>                         | <b>3</b> |
| <b>3 Algoritmi aproximativi</b>                             | <b>4</b> |
| 3.1 Definitii . . . . .                                     | 4        |
| 3.2 Schema demonstratiei de aproximare . . . . .            | 4        |
| <b>4 Knapsack 0/1 - algoritm 1/2-aproximativ</b>            | <b>4</b> |
| 4.1 Problema . . . . .                                      | 4        |
| 4.2 Algoritm . . . . .                                      | 4        |
| 4.3 Ideea demonstratiei . . . . .                           | 4        |
| <b>5 Load Balancing</b>                                     | <b>5</b> |
| 5.1 Problema . . . . .                                      | 5        |
| 5.2 Lower bounds pentru OPT . . . . .                       | 5        |
| 5.3 List Scheduling - factor $2 - 1/m$ . . . . .            | 5        |
| 5.4 Ordered Scheduling - factor $3/2$ . . . . .             | 5        |
| <b>6 TSP</b>                                                | <b>5</b> |
| 6.1 TSP general . . . . .                                   | 5        |
| 6.2 De ce TSP general nu are aproximare constanta . . . . . | 6        |
| 6.3 Metric TSP . . . . .                                    | 6        |
| 6.4 Algoritmul MST / preorder - factor 2 . . . . .          | 6        |
| 6.5 Christofides - factor $3/2$ . . . . .                   | 6        |
| <b>7 Vertex Cover</b>                                       | <b>7</b> |
| 7.1 Definitie . . . . .                                     | 7        |
| 7.2 ApproxVertexCover - factor 2 . . . . .                  | 7        |
| 7.3 Weighted Vertex Cover prin programare liniara . . . . . | 7        |
| <b>8 Algoritmi genetici</b>                                 | <b>7</b> |
| 8.1 Schema generala . . . . .                               | 7        |
| 8.2 Codificare binara . . . . .                             | 8        |
| 8.3 Selectie proportionala - ruleta . . . . .               | 8        |

|           |                                                                  |           |
|-----------|------------------------------------------------------------------|-----------|
| 8.4       | Crossover si mutatie . . . . .                                   | 8         |
| 8.5       | Intrebari teorie posibile la GA . . . . .                        | 8         |
| <b>9</b>  | <b>Algoritmi probabilistici</b>                                  | <b>9</b>  |
| 9.1       | Monte Carlo vs Las Vegas . . . . .                               | 9         |
| 9.2       | Freivalds - verificare $AB = C$ . . . . .                        | 9         |
| 9.3       | Solovay-Strassen . . . . .                                       | 9         |
| 9.4       | Paranoid Quicksort . . . . .                                     | 9         |
| <b>10</b> | <b>Geometrie computationala - formule obligatorii</b>            | <b>9</b>  |
| 10.1      | Testul de orientare . . . . .                                    | 9         |
| 10.2      | Graham scan / Andrew . . . . .                                   | 10        |
| 10.3      | Jarvis march . . . . .                                           | 10        |
| 10.4      | Triangulare . . . . .                                            | 10        |
| 10.5      | Teorema Galeriei de Arta . . . . .                               | 10        |
| 10.6      | Grafuri planare . . . . .                                        | 11        |
| 10.7      | Voronoi . . . . .                                                | 11        |
| 10.8      | Dualitate punct-dreapta . . . . .                                | 11        |
| 10.9      | Semiplane . . . . .                                              | 11        |
| 10.10     | Delaunay si MST . . . . .                                        | 11        |
| <b>11</b> | <b>Ce ar mai putea sa dea ca teorie</b>                          | <b>11</b> |
| 11.1      | Lista de intrebari probabile . . . . .                           | 11        |
| 11.2      | Greseli frecvente . . . . .                                      | 12        |
| <b>II</b> | <b>Rezolvari detaliate - Seminarul 1</b>                         | <b>13</b> |
| <b>12</b> | <b>Seminarul 1 - Probleme computationale, P/NP, greedy si DP</b> | <b>13</b> |
| 12.1      | Exercitiul 1 - exemple de probleme computationale . . . . .      | 13        |
| 12.2      | Exercitiul 2 - decizie vs optimizare . . . . .                   | 13        |
| 12.3      | Exercitiul 3.1 - maximul in valoare absoluta . . . . .           | 13        |
| 12.4      | Exercitiul 3.2 - toate perechile cu suma $S$ . . . . .           | 13        |
| 12.5      | Exercitiul 3.3 - 3SUM . . . . .                                  | 14        |
| 12.6      | Exercitiul 4 - acoperire cu intervale . . . . .                  | 14        |
| <b>13</b> | <b>Problema rucsacului</b>                                       | <b>15</b> |
| 13.1      | Varianta fractionara . . . . .                                   | 15        |
| 13.2      | Varianta discreta 0/1 . . . . .                                  | 15        |
| <b>14</b> | <b>Submultime cu suma data</b>                                   | <b>15</b> |
| <b>15</b> | <b>Impartirea cadourilor</b>                                     | <b>15</b> |

|                                                                     |        |
|---------------------------------------------------------------------|--------|
| 16 Weighted Interval Scheduling                                     | 16     |
| 17 Planificare activitati cu deadline, durata si profit             | 16     |
| 18 Target Sum                                                       | 16     |
| <br>III Rezolvari complete pas cu pas - seminarele lucrate anterior | <br>17 |
| Observatie despre anexele urmatoare                                 | 17     |
| Anexa A - Seminarul 2: algoritmi aproximativi                       | 18     |
| Anexa B - Seminarul 3: algoritmi genetici                           | 36     |
| Anexa C - Seminarul 5: geometrie computationala                     | 65     |
| Anexa D - Seminarul 6: geometrie computationala                     | 80     |
| Anexa E - Seminarul 7: Voronoi, dualitate, semiplane, Delaunay      | 100    |

## Surse folosite

Materialul combina rezolvarile lucrate anterior cu teoria din fisierele de seminar si curs incarcate: Seminarul 1 despre probleme computationale, P/NP, greedy si programare dinamica; Seminarul 2 despre algoritmi aproximativi; Seminarul 3 despre algoritmi genetici; Seminarele 5–7 despre geometrie computationala. Am adaugat si o fisa de teorie din cursurile de algoritmi aproximativi, TSP, Vertex Cover, algoritmi genetici, algoritmi probabilistici si geometrie.

## Part I

# Fisa de teorie pentru examen

## 1 Cum gandesti o problema la examen

La examen, de obicei nu este suficient sa scrii doar algoritmul. Trebuie sa arati:

1. ca solutia produsa este **fezabila**;
2. ce complexitate are;
3. daca este algoritm aproximativ, relatia dintre ALG si OPT;
4. daca este algoritm geometric, ce test de orientare / determinant / formula folosesti;
5. daca este algoritm genetic sau probabilistic, ce inseamna fiecare pas si ce garantie are.

### 1.1 Notatii standard

- $OPT$  = valoarea solutiei optime.
- $ALG$  = valoarea solutiei date de algoritmul propus.
- Problema de **maximizare**: vrem  $ALG$  cat mai mare, deci pentru factor  $\rho \leq 1$  aratam de obicei  $ALG \geq \rho \cdot OPT$ .
- Problema de **minimizare**: vrem  $ALG$  cat mai mic, deci pentru factor  $\rho \geq 1$  aratam de obicei  $ALG \leq \rho \cdot OPT$ .

### 1.2 Complexitati

- $\mathcal{O}(g(n))$  = limita superioara asimptotica. Ex.:  $n^2 + 3n + 7 \in \mathcal{O}(n^2)$ .
- $\Omega(g(n))$  = limita inferioara.
- $\Theta(g(n))$  = limita stransa, si superioara, si inferioara.
- Timp polinomial:  $\mathcal{O}(n^c)$ , unde  $c$  este constanta.
- Pseudo-polinomial: depinde de valoarea numerica a unui parametru, nu doar de lungimea inputului. Exemplu: knapsack discret  $\mathcal{O}(nC)$ .

## 2 P, NP, NP-hard, NP-complet

- $P$  = probleme rezolvabile in timp polinomial de un algoritm determinist.
- $NP$  = probleme pentru care o solutie candidata poate fi verificata in timp polinomial.
- $P \subseteq NP$ ; se presupune ca  $P \neq NP$ .
- O problema este **NP-hard** daca este cel putin la fel de grea ca problemele din  $NP$ ; nu trebuie neaparat sa fie in  $NP$ .
- O problema este **NP-completa** daca este in  $NP$  si este NP-hard.

**Ce poate sa dea teoretic:** "Daca  $A \leq_P B$  si  $B \in P$ , ce stim despre  $A$ ?" Raspuns:  $A \in P$ , deoarece transformam instanta lui  $A$  in instanta lui  $B$  in timp polinomial si apoi rezolvam  $B$  in timp polinomial.

### 3 Algoritmi aproximativi

#### 3.1 Definitii

Pentru probleme NP-hard, deseori renuntam la optimalitate si acceptam o solutie apropiata de optim.

**Minimizare:**

$$\text{OPT}(I) \leq \text{ALG}(I) \leq \rho \cdot \text{OPT}(I), \quad \rho \geq 1.$$

**Maximizare:**

$$\rho \cdot \text{OPT}(I) \leq \text{ALG}(I) \leq \text{OPT}(I), \quad 0 < \rho \leq 1.$$

**Tight bound** inseamna ca factorul nu poate fi imbunatatit pentru acel algoritim: exista intrari pe care raportul ajunge exact la limita demonstrata.

#### 3.2 Schema demonstratiei de aproximare

1. Definesti clar ALG si OPT.
2. Gasesti un lower bound / upper bound pentru OPT.
3. Legi ALG de acel bound.
4. Concluzionezi factorul.

### 4 Knapsack 0/1 - algoritim 1/2-aproximativ

#### 4.1 Problema

Avem obiecte cu valori  $v_i$ , greutati  $g_i$  si capacitate  $G$ . Alegem obiecte intregi, fara fractionare, maximizand valoarea totala.

#### 4.2 Algoritim

```
Sortam obiectele descrescator dupa raportul  $v_i / g_i$ .
S = multimea vida, greutate_curenta = 0.
Pentru fiecare obiect in aceasta ordine:
    daca incape in rucsac, il adaugam in S.
Fie BestItem = obiectul cu valoare maxima care incape singur.
Returnam solutia mai buna dintre S si BestItem.
```

#### 4.3 Ideea demonstratiei

Fie  $\text{OPT}_{0/1}$  valoarea optima pentru varianta discreta si  $\text{OPT}_F$  valoarea optima pentru varianta fractionara. Atunci:

$$\text{OPT}_{0/1} \leq \text{OPT}_F.$$

Greedy pe varianta fractionara ia toate obiectele din  $S$  si eventual o fractie din primul obiect care nu mai incape, notat  $o_k$ . Deci:

$$\text{OPT}_F \leq \text{val}(S) + \text{val}(o_k).$$

Algoritmul returneaza:

$$\text{ALG} = \max\{\text{val}(S), \text{val}(o_k)\}.$$

Cum maximul a doua numere este cel putin jumatate din suma lor:

$$\text{ALG} \geq \frac{\text{val}(S) + \text{val}(o_k)}{2} \geq \frac{\text{OPT}_F}{2} \geq \frac{\text{OPT}_{0/1}}{2}.$$

Deci algoritmul este  $1/2$ -aproximativ.

## 5 Load Balancing

### 5.1 Problema

Avem  $m$  masini identice si  $n$  joburi cu timpi  $t_j$ . Trebuie minimizat makespan-ul:

$$\max_i L_i, \quad L_i = \sum_{j \in J(i)} t_j.$$

### 5.2 Lower bounds pentru OPT

$$\text{OPT} \geq \frac{1}{m} \sum_{j=1}^n t_j, \quad \text{OPT} \geq \max_j t_j.$$

Primul este media incarcarii, al doilea este cel mai lung job, care trebuie pus undeva.

### 5.3 List Scheduling - factor $2 - 1/m$

Algoritm: luam joburile in ordinea data si punem fiecare job pe masina cu load minim curent.

Fie  $q$  ultimul job pus pe masina care ajunge la load maxim. Inainte de  $q$ , acea masina avea load  $\text{load}'(k)$ .

$$\text{ALG} = \text{load}'(k) + t_q.$$

Pentru ca masina  $k$  era cea mai putin incarcata inainte sa primeasca  $q$ :

$$\text{load}'(k) \leq \frac{1}{m} \sum_{j < q} t_j \leq \frac{1}{m} \left( \sum_{j=1}^n t_j - t_q \right).$$

Deci:

$$\text{ALG} \leq \frac{1}{m} \sum_{j=1}^n t_j - \frac{t_q}{m} + t_q = \frac{1}{m} \sum_{j=1}^n t_j + \left(1 - \frac{1}{m}\right) t_q.$$

Folosind lower bound-urile:

$$\text{ALG} \leq \text{OPT} + \left(1 - \frac{1}{m}\right) \text{OPT} = \left(2 - \frac{1}{m}\right) \text{OPT}.$$

### 5.4 Ordered Scheduling - factor $3/2$

Sortam joburile descrescator dupa timp, apoi aplicam List Scheduling. Ideea este ca joburile mari sunt asezate primele, reducand riscul ca un job mare sa ajunga la final pe o masina deja incarcata.

## 6 TSP

### 6.1 TSP general

TSP: graf complet cu ponderi pozitive, trebuie gasit un ciclu Hamiltonian de cost minim. TSP general este NP-hard si nu are algoritm  $c$ -aproximativ pentru niciun  $c$ , presupunand  $P \neq NP$ .



## 6.2 De ce TSP general nu are aproximare constanta

Ideea demonstratiei este reducerea de la Hamiltonian Cycle. Pentru un graf  $G$  cu  $n$  noduri construim un graf complet ponderat  $G'$ :

$$w(e) = \begin{cases} 1, & e \in E(G), \\ cn, & e \notin E(G). \end{cases}$$

Daca  $G$  are ciclu Hamiltonian, atunci  $\text{OPT}(G') = n$ . Un algoritm  $c$ -aproximativ ar da cost cel mult  $cn$ . Daca  $G$  nu are ciclu Hamiltonian, orice tur foloseste macar o muchie de cost  $cn$ , deci costul este strict mai mare decat  $cn$ . Astfel am decide Hamiltonian Cycle in timp polinomial, contradictie daca  $P \neq NP$ .

## 6.3 Metric TSP

Metric TSP este TSP in graf complet ponderat care respecta regula triunghiului:

$$d(u, w) \leq d(u, v) + d(v, w).$$

Aici exista algoritmi aproximativi.

## 6.4 Algoritmul MST / preorder - factor 2

1. Calculam un MST  $T$ .
2. Parcurgem DFS/preorder arborele.
3. Sarim peste nodurile deja vizitate folosind regula triunghiului.
4. Inchidem ciclul.

Avem:

$$\text{MST} \leq \text{OPT}.$$

Dublarea muchiilor MST produce un tur Eulerian de cost  $2\text{MST}$ . Prin scurtcircuitare, costul nu creste datorita regulii triunghiului. Deci:

$$\text{ALG} \leq 2\text{MST} \leq 2\text{OPT}.$$

## 6.5 Christofides - factor 3/2

Algoritmul Christofides pentru Metric TSP:

1. Calculeaza MST  $T$ .
2. Ia multimea  $V^*$  a nodurilor de grad impar din  $T$ .
3. Calculeaza un cuplaj perfect minim  $M$  pe  $V^*$ .
4. Reuneste  $T \cup M$ ; toate gradele devin pare.
5. Gaseste un ciclu Eulerian.
6. Scurtcircuiteaza nodurile repetate.

Garantia este:

$$\text{cost}(T) \leq \text{OPT}, \quad \text{cost}(M) \leq \frac{1}{2}\text{OPT},$$

deci:

$$\text{ALG} \leq \text{cost}(T) + \text{cost}(M) \leq \frac{3}{2}\text{OPT}.$$

## 7 Vertex Cover

### 7.1 Definitie

Pentru un graf neorientat  $G = (V, E)$ , un vertex cover este o multime  $S \subseteq V$  astfel incat pentru orice muchie  $(u, v) \in E$ , avem  $u \in S$  sau  $v \in S$ .

### 7.2 ApproxVertexCover - factor 2

```

S = multimea vida
E' = E
cat timp E' nu este vida:
    alegem o muchie (u,v) din E'
    adaugam u si v in S
    stergem toate muchiile incidente cu u sau v
return S

```

Muchiile alese sunt doua cate doua disjuncte in varfuri. Fie  $E^*$  multimea lor. Orice vertex cover optim trebuie sa contina macar un capat din fiecare muchie din  $E^*$ , deci:

$$OPT \geq |E^*|.$$

Algoritmul adauga doua varfuri pentru fiecare muchie aleasa:

$$|S| = 2|E^*| \leq 2OPT.$$

Deci este 2-aproximativ.

### 7.3 Weighted Vertex Cover prin programare liniara

Formulare intrega:

$$\min \sum_{v \in V} w_v x_v$$

cu restrictii:

$$x_u + x_v \geq 1, \quad \forall (u, v) \in E, \quad x_v \in \{0, 1\}.$$

Relaxarea LP permite:

$$x_v \in [0, 1].$$

Rounding standard:

$$S = \{v \in V \mid x_v \geq 1/2\}.$$

Este 2-aproximativ, deoarece fiecare muchie ramane acoperita si costul rotunjit este cel mult dublul costului solutiei LP, iar LP este lower bound pentru optimul intreg.

## 8 Algoritmi genetici

### 8.1 Schema generala

Un algoritm genetic este o metaheuristica pentru optimizare. Nu garanteaza optimul, dar cauta solutii bune intr-un spatiu mare.

```

Genereaza populatia initiala P(0)
Pentru fiecare generatie t:
    evalueaza fitness-ul fiecarui individ
    selecteaza indivizi pentru reproducere

```

```

    aplica crossover cu probabilitate pc
    aplica mutatie cu probabilitate pm
    optional: pastreaza elitist cel mai bun individ
Returneaza cel mai bun individ gasit

```

## 8.2 Codificare binara

Pentru domeniul  $[a, b]$  si precizie  $p$  zecimale:

$$\ell = \lceil \log_2 ((b - a)10^p) \rceil.$$

Valoarea intreaga a cromozomului:

$$v = \sum_{k=1}^{\ell} g_k 2^{\ell-k}.$$

Decodificare:

$$x = a + v \cdot \frac{b - a}{2^{\ell} - 1}.$$

Codificare inversa:

$$v = \text{round} \left( (x - a) \frac{2^{\ell} - 1}{b - a} \right).$$

## 8.3 Selectie proportionala - ruleta

$$p_i = \frac{f(X_i)}{\sum_j f(X_j)}, \quad q_i = p_1 + p_2 + \dots + p_i.$$

Generam  $u \in [0, 1)$  si alegem  $X_i$  cu:

$$q_{i-1} \leq u < q_i.$$

## 8.4 Crossover si mutatie

- Crossover cu un punct: se taie la pozitia  $k$ , primul copil ia prefix de la parintele 1 si sufix de la parintele 2.
- Crossover cu doua puncte: se inverseaza segmentul dintre  $k_1$  si  $k_2$ .
- Uniform: o masca decide din care parinte se ia fiecare gena.
- Mutatie rara: cu probabilitate  $p_m$  se schimba o gena aleatoare.
- Mutatie per gena: fiecare gena se completeaza independent cu probabilitate  $p_m$ .

## 8.5 Intrebari teorie posibile la GA

- De ce se foloseste mutatia? Pentru diversitate si evitarea convergentei premature.
- Ce face elitismul? Pastreaza cel mai bun individ, dar poate accelera convergenta prematura.
- Ce se intampla daca  $p_c = 0$ ? Nu se combina informatia; avem cautari locale paralele prin mutatii.
- Ce se intampla daca  $p_m$  e prea mare? Cautarea devine aproape aleatoare.
- Ce se intampla daca  $p_m$  e prea mic? Populatia poate ramane blocata intr-un optim local.

## 9 Algoritmi probabilistici

### 9.1 Monte Carlo vs Las Vegas

- Monte Carlo: timp controlat / polinomial, dar raspuns probabil corect. Exemplu: Freivalds, Solovay-Strassen.
- Las Vegas: raspuns mereu corect, dar timpul de rulare este aleator. Exemplu: randomized quicksort/paranoid quicksort.

### 9.2 Freivalds - verificare $AB = C$

Alegem vector aleator binar  $r \in \{0, 1\}^n$ . Verificam:

$$A(Br) = Cr.$$

Daca  $AB = C$ , algoritmul raspunde mereu DA. Daca  $AB \neq C$ , atunci probabilitatea sa detecteze eroarea intr-o iteratie este cel putin  $1/2$ :

$$\Pr[A(Br) \neq Cr] \geq \frac{1}{2}.$$

Dupa  $k$  repetari, probabilitatea de fals pozitiv este cel mult:

$$\left(\frac{1}{2}\right)^k.$$

Complexitatea unei iteratii este  $\mathcal{O}(n^2)$ .

### 9.3 Solovay-Strassen

Test probabilistic de primalitate. Pentru  $n$  impar, alegem  $a$  aleator si comparam simbolul Jacobi cu criteriul lui Euler:

$$\left(\frac{a}{n}\right) \equiv a^{(n-1)/2} \pmod{n}.$$

Daca nu se verifica,  $n$  este compus. Daca se verifica,  $n$  este probabil prim. Pentru compus, probabilitatea de fals pozitiv pe o iteratie este cel mult  $1/2$ .

### 9.4 Paranoid Quicksort

Pivot bun: partiile  $L$  si  $G$  au dimensiune cel mult  $3n/4$ . Probabilitatea unui pivot bun este cel putin  $1/2$ , deci in medie alegem 2 pivoti pana gasim unul bun. Recurenta este de forma:

$$T(n) \leq T(|L|) + T(|G|) + cn, \quad |L|, |G| \leq \frac{3n}{4}.$$

Rezultatul este  $\mathcal{O}(n \log n)$  in medie / expected.

## 10 Geometrie computationala - formule obligatorii

### 10.1 Testul de orientare

Pentru  $P = (p_x, p_y)$ ,  $Q = (q_x, q_y)$ ,  $R = (r_x, r_y)$ :

$$\Delta(P, Q, R) = \begin{vmatrix} 1 & 1 & 1 \\ p_x & q_x & r_x \\ p_y & q_y & r_y \end{vmatrix}.$$

Echivalent:

$$\Delta = (q_x - p_x)(r_y - p_y) - (q_y - p_y)(r_x - p_x).$$

Interpretare:

$$\Delta > 0 \Rightarrow R \text{ este la stanga lui } \overrightarrow{PQ},$$

$$\Delta < 0 \Rightarrow R \text{ este la dreapta lui } \overrightarrow{PQ},$$

$$\Delta = 0 \Rightarrow P, Q, R \text{ sunt coliniare.}$$

## 10.2 Graham scan / Andrew

Sortam punctele dupa  $x$ , apoi dupa  $y$ . Construim marginea inferioara si superioara. Pentru marginea inferioara, cat timp ultimele 3 puncte nu fac viraj la stanga, eliminam punctul din mijloc.

```
L = lista vida
pentru punctele p in ordine crescatoare:
    adauga p in L
    cat timp ultimele 3 puncte nu fac viraj la stanga:
        elimina punctul din mijloc dintre ultimele 3
```

Complexitate:

$$\mathcal{O}(n \log n)$$

din sortare; partea cu stiva este liniara.

## 10.3 Jarvis march

Pornim de la un punct extrem si alegem succesiv punctul cel mai din dreapta / stanga fata de muchia curenta, in functie de sensul dorit. Complexitate:

$$\mathcal{O}(nh),$$

unde  $h$  este numarul de puncte de pe acoperirea convexa.

## 10.4 Triangulare

Pentru o multime de  $n$  puncte in plan, cu  $k$  puncte pe frontiera acoperirii convexe:

$$n_t = 2n - k - 2, \quad n_m = 3n - k - 3.$$

Aici  $n_t$  este numarul de triunghiuri, iar  $n_m$  numarul de muchii din triangulare.

Pentru un poligon simplu cu  $n$  varfuri:

$$\text{numar triunghiuri} = n - 2, \quad \text{numar diagonale} = n - 3.$$

## 10.5 Teorema Galeriei de Arta

Orice poligon simplu cu  $n$  varfuri poate fi supravegheat cu:

$$\left\lfloor \frac{n}{3} \right\rfloor$$

camere, iar aceasta limita este stransa. Metoda: triangulam poligonul, 3-coloram varfurile, alegem culoarea cu cele mai putine varfuri.

## 10.6 Grafuri planare

Formula lui Euler pentru graf planar conex:

$$v - m + f = 2.$$

Daca fiecare fata are grad cel putin 3:

$$2m \geq 3f \Rightarrow f \leq \frac{2m}{3}.$$

Daca fiecare varf are grad cel putin 3:

$$2m \geq 3v \Rightarrow v \leq \frac{2m}{3}.$$

Pentru graf planar simplu cu  $v \geq 3$ :

$$m \leq 3v - 6.$$

## 10.7 Voronoi

Pentru  $n$  situri necoliniare:

$$n_v \leq 2n - 5, \quad n_m \leq 3n - 6.$$

Numarul de semidrepte ale diagramei Voronoi este egal cu numarul de puncte de pe frontiera acoperirii convexe.

## 10.8 Dualitate punct-dreapta

Conventia folosita:

$$p = (p_x, p_y) \quad \longleftrightarrow \quad p^* : y = p_x x - p_y.$$

Pentru dreapta neverticala:

$$d : y = mx + n \quad \longleftrightarrow \quad d^* = (m, -n).$$

Incidenta se inverseaza: daca punctul  $p$  este pe dreapta  $d$ , atunci punctul  $d^*$  este pe dreapta  $p^*$ .

## 10.9 Semiplane

Daca normala exterioara este coliniara cu  $(a, b, c)$ , atunci semiplanul standard este:

$$ax + by + c \leq 0.$$

## 10.10 Delaunay si MST

Arborele partial de cost minim pe o multime de puncte este subgraf al triangulatiei Delaunay. Ideea: daca o muchie  $uv$  din MST nu este Delaunay, atunci exista o muchie mai scurta care ar putea inlocui  $uv$ , contradictie cu minimalitatea MST.

# 11 Ce ar mai putea sa dea ca teorie

## 11.1 Lista de intrebari probabile

1. Definiti  $P$ ,  $NP$ , NP-hard, NP-complet.

2. Explicati diferenta dintre problema de decizie si problema de optimizare.
3. Ce inseamna algoritmul  $\rho$ -aproximativ pentru minimizare / maximizare?
4. Ce inseamna tight bound?
5. Demonstrati  $\text{OPT}_{0/1} \leq \text{OPT}_F$  pentru knapsack.
6. Demonstrati factorul  $1/2$  pentru algoritmul greedy modificat de knapsack.
7. Demonstrati lower bound-urile pentru Load Balancing.
8. Demonstrati ca List Scheduling este  $2 - 1/m$  aproximativ.
9. De ce TSP general nu are aproximare constanta daca  $P \neq NP$ ?
10. Ce este Metric TSP si de ce regula triunghiului conteaza?
11. Demonstrati ca algoritmul cu MST pentru Metric TSP este 2-aproximativ.
12. Ce adauga Christofides fata de algoritmul MST?
13. Demonstrati ApproxVertexCover 2-aproximativ.
14. Scrieti formularea LP pentru Weighted Vertex Cover si ideea de rounding.
15. Explicati schema unui algoritmul genetic.
16. Cum se calculeaza lungimea cromozomului pentru  $[a, b]$  si precizie  $p$ ?
17. Explicati ruleta, turneul, crossover-ul si mutatia.
18. Diferenta Monte Carlo vs Las Vegas.
19. Demonstrati garantia de eroare la Freivalds.
20. Ce este un pivot bun la Paranoid Quicksort?
21. Scrieti determinantul de orientare si interpretarile semnului.
22. Explicati Andrew / Graham scan pas cu pas.
23. Formulele pentru triangulare:  $2n - k - 2$  si  $3n - k - 3$ .
24. Teorema Galeriei de Arta: enunt si metoda prin triangulare + 3-colorare.
25. Formula lui Euler pentru grafuri planare si consecintele.
26. Legatura Voronoi - acoperire convexa: semidrepte = puncte pe hull.
27. Dualitatea punct-dreapta si proprietatea de incidenta.
28. De ce MST este subgraf al triangulatiei Delaunay?

## 11.2 Greseli frecvente

- La probleme de maximizare se scrie gresit  $\text{ALG} \leq \rho \text{OPT}$ . Pentru maximizare vrem  $\text{ALG} \geq \rho \text{OPT}$ .
- La Load Balancing, masina cu load maxim se analizeaza prin ultimul job adaugat.
- La TSP metric, muchia care nu este in MST este justificata prin scurtcircuitare si regula triunghiului.
- La Andrew, se elimina punctul din mijloc al ultimelor trei puncte daca nu apare virajul dorit.
- La triangulare,  $k$  este numarul de puncte de pe frontiera hull-ului, nu numarul de varfuri ale poligonului daca exista puncte interioare.
- La Voronoi, numarul de semidrepte nu este intotdeauna  $n$ , ci numarul de situri de pe acoperirea convexa.

## Part II

# Rezolvări detaliate - Seminarul 1

## 12 Seminarul 1 - Probleme computationale, P/NP, greedy și DP

### 12.1 Exercițiul 1 - exemple de probleme computationale

O problema computatională este o problemă pentru care putem descrie clar datele de intrare, datele de ieșire și un algoritm care produce răspunsul.

**Exemple din viața de zi cu zi:**

1. **Alegerea traseului până la facultate.** Input: harta, timpii pe drumuri, punctul de start și destinația. Output: traseul cu timp minim. Este o problemă de optimizare.
2. **Planificarea învățatului pentru examene.** Input: lista materiilor, timpul disponibil, dificultatea fiecărei materii. Output: un program care maximizează pregătirea sau minimizează riscul de restanță.

### 12.2 Exercițiul 2 - decizie vs optimizare

**Probleme de decizie:**

- Există drum de la nodul  $s$  la nodul  $t$  într-un graf?
- Se poate acoperi intervalul  $[a, b]$  cu intervalele disponibile?

**Probleme de optimizare:**

- Care este drumul minim de la  $s$  la  $t$ ?
- Care este numărul minim de intervale necesare pentru a acoperi  $[a, b]$ ?

### 12.3 Exercițiul 3.1 - maximul în valoare absolută

Algoritm: parcurgem vectorul o singură dată și reținem elementul cu  $|x|$  maxim.

```
best = a[1]
for i = 2..n:
    if abs(a[i]) > abs(best):
        best = a[i]
return best
```

Complexitate timp:

$$\mathcal{O}(n),$$

fiindcă verificăm fiecare element o dată. Memorie:

$$\mathcal{O}(1),$$

fiindcă reținem doar variabila  $best$ .

### 12.4 Exercițiul 3.2 - toate perechile cu suma $S$

Metoda eficientă pentru a decide dacă există perechi: hash set.



```

vazute = multime vida
pentru fiecare x din vector:
    daca S-x este in vazute:
        am gasit o pereche
    adauga x in vazute

```

Pentru listarea tuturor perechilor distincte ca valori, putem sorta si folosi doi pointeri:

$$\mathcal{O}(n \log n).$$

Daca se cer toate perechile de pozitii, in cel mai rau caz output-ul poate fi  $\Theta(n^2)$ , deci nu putem avea timp mai bun decat dimensiunea raspunsului.

## 12.5 Exerciitiul 3.3 - 3SUM

Algoritm determinist standard:

1. Sortam vectorul:  $\mathcal{O}(n \log n)$ .
2. Pentru fiecare  $i$ , cautam doua numere cu suma  $-a_i$  folosind doi pointeri.

Complexitate:

$$\mathcal{O}(n^2).$$

Nedeterminist, putem “ghici” trei pozitii si verifica in timp  $\mathcal{O}(1)$  daca suma este 0. De aceea problema este in NP.

## 12.6 Exerciitiul 4 - acoperire cu intervale

Avem interval tinta  $[a, b]$  si intervale disponibile  $[a_i, b_i]$ . Vrem numarul minim de intervale care acopera complet  $[a, b]$ .

**Algoritm greedy optim:**

```

sortam intervalele crescator dupa capatul stang
curent = a
solutie = lista vida
cat timp curent < b:
    dintre intervalele cu start <= curent alegem intervalul cu end maxim
    daca nu exista un asemenea interval: return imposibil
    adaugam intervalul ales in solutie
    curent = end-ul intervalului ales
return solutie

```

**Justificare optimala:** La primul pas, orice solutie valida trebuie sa aleaga un interval care incepe cel tarziu in  $a$ . Alegerea greedy este intervalul care ajunge cel mai departe. Daca o solutie optima a ales alt interval, il putem inlocui cu intervalul greedy fara sa folosim mai multe intervale si fara sa acoperim mai putin. Repetam argumentul pentru restul intervalului ramas. Deci greedy este optim.

Complexitate:

$$\mathcal{O}(n \log n)$$

datorita sortarii.

## 13 Problema rucsacului

### 13.1 Varianta fractionara

Sortam obiectele descrescator dupa raportul:

$$\frac{val_i}{g_i}.$$

Luam obiecte in aceasta ordine, iar ultimul poate fi luat fractionar.

Formula pentru contributia unui obiect fractionar:

$$p_i val_i, \quad p_i g_i, \quad p_i \in [0, 1].$$

Greedy este optim pentru varianta fractionara, pentru ca intotdeauna alegem cea mai buna valoare pe unitate de greutate.

### 13.2 Varianta discreta 0/1

Programare dinamica:

$$dp[i][w] = \text{valoarea maxima folosind primele } i \text{ obiecte si capacitate } w.$$

Recurenta:

$$\begin{aligned} dp[i][w] &= dp[i-1][w], \quad \text{daca } g_i > w, \\ dp[i][w] &= \max\{dp[i-1][w], dp[i-1][w-g_i] + val_i\}, \quad \text{daca } g_i \leq w. \end{aligned}$$

Complexitate:

$$\mathcal{O}(nC).$$

## 14 Submultime cu suma data

Definim:

$$dp[i][s] = \begin{cases} 1, & \text{daca putem obtine suma } s \text{ cu primele } i \text{ elemente,} \\ 0, & \text{altfel.} \end{cases}$$

Recurenta:

$$dp[i][s] = dp[i-1][s] \vee dp[i-1][s-a_i].$$

Complexitate:

$$\mathcal{O}(nM).$$

## 15 Impartirea cadourilor

Fie  $S$  suma totala. Vrem doua submultimi cu sume cat mai apropiate. Cautam cea mai mare suma  $x \leq S/2$  care se poate obtine prin subset sum. Rezultatul este:

$$x \quad \text{si} \quad S - x.$$

Diferenta este:

$$S - 2x.$$

## 16 Weighted Interval Scheduling

Sortam activitatile dupa timpul de final. Definim  $p(i)$  = ultimul interval compatibil cu  $i$  inaintea lui  $i$ .

Recurenta:

$$dp[i] = \max\{dp[i-1], profit_i + dp[p(i)]\}.$$

Cu cautare binara pentru  $p(i)$ :

$$\mathcal{O}(n \log n).$$

## 17 Planificare activitati cu deadline, durata si profit

Sortam activitatile dupa deadline. Definim:

$$dp[i][t] = \text{profit maxim folosind primele } i \text{ activitati si timp total } t.$$

Pentru activitatea  $i$  cu durata  $l_i$ , deadline  $t_i$  si profit  $p_i$ :

$$dp[i][t] = \max(dp[i-1][t], dp[i-1][t-l_i] + p_i),$$

cu conditia ca activitatea sa se termine pana la deadline. Complexitate:

$$\mathcal{O}(nT + n \log n), \quad T = \max_i t_i.$$

## 18 Target Sum

Vrem semne  $s_i \in \{-1, 1\}$  astfel incat:

$$\sum_{i=1}^n s_i a_i = T.$$

Notam  $P$  suma numerelor cu plus si  $N$  suma numerelor cu minus. Avem:

$$P - N = T, \quad P + N = S.$$

Adunand:

$$2P = T + S \Rightarrow P = \frac{T + S}{2}.$$

Deci problema devine: cate submultimi au suma  $P$ ? Daca  $T + S$  este impar sau  $|T| > S$ , raspunsul este 0.

## Part III

# Rezolvări complete pas cu pas - seminarele lucrate anterior

## Observatie despre anexele urmatoare

Urmatoarele anexe includ rezolvarile complete produse anterior, pastrate integral in forma lor compilata. Inaintea lor a fost adaugata fisa de teorie, astfel incat documentul sa poata fi folosit direct pentru examen.

## **Anexa A - Seminarul 2: algoritmi aproximativi**

# Seminar 2 – Algoritmi aproximativi

## Rezolvare detaliată pentru examen

### Ideea generală pentru examen

În acest seminar apar probleme de optimizare pentru care nu cerem neapărat soluția optimă, ci o soluție apropiată de optim.

Pentru o problemă de **maximizare**, un algoritm este  $\alpha$ -aproximativ, cu  $0 < \alpha \leq 1$ , dacă pentru orice instanță avem:

$$\text{ALG} \geq \alpha \cdot \text{OPT}.$$

De exemplu, un algoritm  $\frac{1}{2}$ -aproximativ garantează:

$$\text{ALG} \geq \frac{1}{2} \text{OPT}.$$

Pentru o problemă de **minimizare**, un algoritm este  $\rho$ -aproximativ, cu  $\rho \geq 1$ , dacă pentru orice instanță avem:

$$\text{ALG} \leq \rho \cdot \text{OPT}.$$

De exemplu, un algoritm 2-aproximativ garantează:

$$\text{ALG} \leq 2 \text{OPT}.$$

# 1 Problema 1 – Vouchere de vacanță

## Enunț pe scurt

George are o sumă  $S$  pe cardul de vouchere. Există  $n$  locații, iar locația  $i$  costă  $C_i$ . George vrea să aleagă o submulțime de locații astfel încât suma costurilor să nu depășească  $S$ , dar să fie cât mai mare.

Matematic, vrem să alegem o mulțime  $L \subseteq \{1, 2, \dots, n\}$  astfel încât:

$$\sum_{i \in L} C_i \leq S$$

și suma:

$$\sum_{i \in L} C_i$$

să fie maximă.

Aceasta este, practic, o variantă de **0/1 Knapsack** în care valoarea fiecărei locații este egală cu greutatea/costul ei:

$$valoare(i) = C_i, \quad greutate(i) = C_i.$$

## Observație importantă

Dacă o locație are costul mai mare decât suma disponibilă, adică:

$$C_i > S,$$

atunci acea locație nu poate fi aleasă niciodată, deci o ignorăm.

Presupunem deci, după filtrare, că:

$$C_i \leq S, \quad \forall i.$$

## Algoritm $\frac{1}{2}$ -aproximativ

Vom construi o soluție greedy și o comparăm cu o locație individuală.

### Algoritm:

1. Eliminăm toate locațiile cu  $C_i > S$ .
2. Inițializăm:

$$A = \emptyset, \quad suma(A) = 0.$$

3. Parcurgem locațiile rămase, în orice ordine.
4. Pentru fiecare locație  $i$ :

- dacă  $\text{suma}(A) + C_i \leq S$ , adăugăm locația:

$$A \leftarrow A \cup \{i\};$$

- altfel, ne oprim și notăm cu  $j$  prima locație care nu mai încapă.

5. Dacă nu există o astfel de locație  $j$ , atunci  $A$  conține toate locațiile posibile și este optimă.
6. Dacă există  $j$ , alegem soluția mai bună dintre:

$$A$$

și soluția formată doar din locația  $j$ :

$$\{j\}.$$

Valoarea soluției algoritmului este:

$$\text{ALG} = \max\{\text{suma}(A), C_j\}.$$

## Pseudocod

**ApproxVouchere**( $S, C_1, \dots, C_n$ )

$A \leftarrow \emptyset$

$\text{sum} \leftarrow 0$

pentru fiecare locație  $i$  cu  $C_i \leq S$ :

    dacă  $\text{sum} + C_i \leq S$ :

$A \leftarrow A \cup \{i\}$

$\text{sum} \leftarrow \text{sum} + C_i$

    altfel:

$j \leftarrow i$

        oprește parcurgerea

dacă nu există  $j$ :

    returnează  $(A, \text{sum})$

dacă  $\text{sum} \geq C_j$ :

    returnează  $(A, \text{sum})$

altfel:

    returnează  $(\{j\}, C_j)$

## Complexitate

Parcurgem fiecare locație cel mult o dată. Deci:

$$T(n) = O(n).$$

Dacă alegem să sortăm locațiile înainte, complexitatea ar deveni:

$$O(n \log n),$$

dar sortarea nu este necesară pentru demonstrația de  $\frac{1}{2}$ -aproximare.



## Demonstrația factorului de aproximare

Notăm:

$$A_s = \text{suma}(A).$$

Locația  $j$  este prima locație care nu mai încapă în rucsacul de vouchere. Asta înseamnă:

$$A_s + C_j > S.$$

Soluția optimă nu poate avea valoare mai mare decât suma totală disponibilă:

$$\text{OPT} \leq S.$$

Din cele două relații rezultă:

$$A_s + C_j > S \geq \text{OPT}.$$

Deci:

$$A_s + C_j > \text{OPT}.$$

Algoritmul alege maximul dintre  $A_s$  și  $C_j$ :

$$\text{ALG} = \max\{A_s, C_j\}.$$

Pentru orice două numere pozitive  $x$  și  $y$ , avem:

$$\max\{x, y\} \geq \frac{x + y}{2}.$$

Aplicăm pentru  $x = A_s$  și  $y = C_j$ :

$$\text{ALG} = \max\{A_s, C_j\} \geq \frac{A_s + C_j}{2}.$$

Dar știm că:

$$A_s + C_j > \text{OPT}.$$

Prin urmare:

$$\frac{A_s + C_j}{2} > \frac{\text{OPT}}{2}.$$

Deci:

$$\text{ALG} \geq \frac{1}{2} \text{OPT}.$$

**Concluzie:** algoritmul este  $\frac{1}{2}$ -aproximativ.

## Ce trebuie reținut pentru examen

Pentru problema de vouchere trebuie să știi schema:

greedy parțial + primul obiect care nu încapă.

Demonstrația se bazează pe ideea:

$$A_s + C_j > S \geq \text{OPT}.$$

Apoi alegem maximul dintre cele două părți:

$$\max\{A_s, C_j\} \geq \frac{A_s + C_j}{2} > \frac{\text{OPT}}{2}.$$

## 2 Problema 2 – Camioane și transporturi

### Enunț pe scurt

Avem  $n$  colete, cu greutate:

$$w_1, w_2, \dots, w_n.$$

Fiecare camion are capacitate:

$$G.$$

Presupunem că fiecare colet încapă individual într-un camion:

$$w_i \leq G, \quad \forall i.$$

Vrem să minimizăm numărul de camioane folosite.

Algoritmul din enunț este de tip **Next Fit**:

- încărcăm camionul curent cu colete în ordinea dată;
- dacă următorul colet nu mai încapă, închidem camionul curent;
- deschidem un camion nou și punem acolo coletul care nu a încăput.

### 2.1 a) Exemplu în care algoritmul nu este optim

Luăm:

$$G = 10$$

și coletele, în această ordine:

$$W = [7, 5, 3, 5].$$

#### Soluția algoritmului

Camionul 1:

$$7$$

Următorul colet are greutate 5, dar:

$$7 + 5 = 12 > 10,$$

deci camionul 1 se închide.

Camionul 2:

$$5 + 3 = 8.$$

Următorul colet are greutate 5, dar:

$$8 + 5 = 13 > 10,$$

deci camionul 2 se închide.

Camionul 3:

5.

Algoritmul folosește:

$$\text{ALG} = 3 \text{ camioane.}$$

### Soluția optimă

Putem grupa coletele astfel:

$$7 + 3 = 10,$$

$$5 + 5 = 10.$$

Deci soluția optimă folosește:

$$\text{OPT} = 2 \text{ camioane.}$$

Prin urmare, algoritmul nu este mereu optim, deoarece aici:

$$\text{ALG} = 3 > 2 = \text{OPT}.$$

## 2.2 b) Demonstrație că algoritmul este 2-aproximativ

Vrem să arătăm:

$$\text{ALG} \leq 2 \text{ OPT}.$$

Notăm cu:

$$L_i$$

încărcătura camionului  $i$  în soluția produsă de algoritm.

Deci:

$$L_i \leq G, \quad \forall i.$$

Fie:

$$\text{ALG} = k.$$

Asta înseamnă că algoritmul a folosit  $k$  camioane.

### Observație-cheie

Oricare două camioane consecutive au împreună încărcătură strict mai mare decât  $G$ :

$$L_i + L_{i+1} > G.$$

**De ce?**

Camionul  $i$  a fost închis exact pentru că primul colet din camionul  $i + 1$  nu mai încăpea în camionul  $i$ .

Dacă notăm acel colet cu greutatea  $x$ , atunci:

$$L_i + x > G.$$

Dar camionul  $i + 1$  conține coletul  $x$ , posibil și alte colete, deci:

$$L_{i+1} \geq x.$$

Rezultă:

$$L_i + L_{i+1} \geq L_i + x > G.$$

Așadar:

$$L_i + L_{i+1} > G.$$

## Folosim observația pentru a demonstra aproximarea

Împerechem camioanele două câte două:

$$(1, 2), (3, 4), (5, 6), \dots$$

Pentru fiecare pereche avem:

$$L_{2r-1} + L_{2r} > G.$$

Dacă algoritmul folosește  $k$  camioane, atunci avem cel puțin:

$$\left\lfloor \frac{k}{2} \right\rfloor$$

perechi complete.

Fie  $T$  greutatea totală a coletelor:

$$T = \sum_{i=1}^n w_i.$$

Din observația de mai sus:

$$T > \left\lfloor \frac{k}{2} \right\rfloor G.$$

Orice soluție, inclusiv cea optimă, trebuie să transporte greutatea totală  $T$ . Cum fiecare camion poate transporta cel mult  $G$ , avem lower bound-ul:

$$\text{OPT} \geq \frac{T}{G}.$$

Din:

$$T > \left\lfloor \frac{k}{2} \right\rfloor G$$

rezultă:

$$\frac{T}{G} > \left\lfloor \frac{k}{2} \right\rfloor.$$

Deci:

$$\text{OPT} > \left\lfloor \frac{k}{2} \right\rfloor.$$

Cum OPT este număr întreg, rezultă:

$$\text{OPT} \geq \left\lfloor \frac{k}{2} \right\rfloor + 1.$$

Această relație implică faptul că  $k$  nu poate fi mai mare decât  $2 \text{OPT}$ . Astfel:

$$\text{ALG} = k \leq 2 \text{OPT}.$$

**Concluzie:** algoritmul Next Fit este 2-aproximativ.

## Variantă scurtă de reținut

Ideea de examen este:

1. Camionul curent se închide doar când următorul colet nu mai încapă.
2. Deci două camioane consecutive au încărcătură totală mai mare decât capacitatea unui camion:

$$L_i + L_{i+1} > G.$$

3. Asta înseamnă că algoritmul nu poate irosi prea mult spațiu.
4. Prin împerecherea camioanelor rezultă:

$$\text{ALG} \leq 2 \text{OPT}.$$

### 3 Problema 3 – Cadourile lui Moș Crăciun

#### Enunț pe scurt

Avem:

- $m$  copii;
- $n$  cadouri;
- cadoul  $i$  are valoarea  $val(i)$ .

Valoarea totală a cadourilor este:

$$\text{Val} = \sum_{i=1}^n val(i).$$

Trebuie să împărțim cadourile astfel încât copilul care primește cel mai puțin să primească o valoare cât mai mare.

Pentru copilul  $k$ , notăm cu:

$$W(k)$$

valoarea totală primită de copilul  $k$ .

Scopul este să maximizăm:

$$\min_{1 \leq k \leq m} W(k).$$

Deci:

$$\text{OPT} = \max_{\text{împărțiri}} \min_{1 \leq k \leq m} W(k).$$

Restricția importantă din enunț este:

$$val(i) \leq \frac{\text{Val}}{2m}, \quad \forall i.$$

#### 3.1 a) Algoritm $\frac{1}{2}$ -aproximativ în $O(n \log m)$

Folosim o strategie greedy: fiecare cadou este dat copilului care are, în acel moment, valoarea totală minimă.

Pentru a găsi rapid copilul cu suma minimă, folosim un **min-heap** cu  $m$  elemente.

**Algoritm:**

1. Inițial, fiecare copil are suma 0:

$$W(1) = W(2) = \dots = W(m) = 0.$$

2. Punem cei  $m$  copii într-un min-heap după valoarea  $W(k)$ .

3. Pentru fiecare cadou  $i = 1, 2, \dots, n$ :

- a) extragem din heap copilul  $k$  cu valoare minimă  $W(k)$ ;
- b) îi dăm cadoul  $i$ ;
- c) actualizăm:

$$W(k) \leftarrow W(k) + val(i);$$

- d) introducem copilul  $k$  înapoi în heap.

4. La final, soluția algoritmului este:

$$ALG = \min_{1 \leq k \leq m} W(k).$$

## Pseudocod

**GreedyCadouri**( $m, val(1), \dots, val(n)$ )  
pentru fiecare copil  $k = 1, 2, \dots, m$ :  
     $W(k) \leftarrow 0$   
     $C(k) \leftarrow \emptyset$   
construiește un min-heap cu perechi  $(W(k), k)$   
pentru fiecare cadou  $i = 1, 2, \dots, n$ :  
    extrage copilul  $k$  cu valoare minimă  $W(k)$   
     $C(k) \leftarrow C(k) \cup \{i\}$   
     $W(k) \leftarrow W(k) + val(i)$   
    inserează înapoi  $(W(k), k)$  în min-heap  
returnează  $C(1), C(2), \dots, C(m)$  și  $\min_k W(k)$

## Complexitate

Pentru fiecare cadou facem:

- o extragere din heap:  $O(\log m)$ ;
- o inserare în heap:  $O(\log m)$ .

Pentru  $n$  cadouri:

$$T(n, m) = O(n \log m).$$

### 3.2 b) Relația dintre $W(k)$ și $W(q) - val(i)$

Fie  $k$  copilul care primește cel mai puțin la final, deci:

$$ALG = W(k).$$

Fie  $q$  un copil oarecare, cu  $q \neq k$ . Fie  $i$  ultimul cadou primit de copilul  $q$ .

Atunci, înainte ca  $q$  să primească ultimul său cadou  $i$ , copilul  $q$  avea valoarea:

$$W(q) - val(i).$$



Algoritmul dă mereu următorul cadou copilului cu suma minimă la acel moment. Deci, în momentul în care copilul  $q$  a primit cadoul  $i$ , suma lui era cel mult suma oricărui alt copil, inclusiv a copilului  $k$  în acel moment.

Notăm cu  $W_t(k)$  suma copilului  $k$  în acel moment. Avem:

$$W(q) - val(i) \leq W_t(k).$$

Dar sumele copiilor cresc doar prin adăugare de cadouri, deci:

$$W_t(k) \leq W(k).$$

Prin urmare:

$$W(q) - val(i) \leq W(k).$$

Cum  $W(k) = \text{ALG}$ , obținem:

$$\boxed{W(q) - val(i) \leq \text{ALG} .}$$

Echivalent:

$$\boxed{W(q) \leq \text{ALG} + val(i) .}$$

Aceasta este relația-cheie pentru demonstrație.

### 3.3 c) Demonstrație că $\text{ALG} \geq \frac{\text{Val}}{2m}$

Pentru fiecare copil  $q$ , notăm cu  $i_q$  ultimul cadou primit de acel copil. Acest lucru are sens deoarece  $n \geq m$ , iar algoritmul dă primele  $m$  cadouri copiilor cu sumă 0, deci fiecare copil primește cel puțin un cadou.

Din punctul b), pentru orice copil  $q \neq k$ , avem:

$$W(q) - val(i_q) \leq \text{ALG} .$$

Pentru copilul  $k$ , relația este trivial adevărată:

$$W(k) - val(i_k) \leq W(k) = \text{ALG} .$$

Deci, pentru orice copil  $q$ :

$$W(q) - val(i_q) \leq \text{ALG} .$$

Rezultă:

$$W(q) \leq \text{ALG} + val(i_q) .$$

Sumăm după toți copiii:

$$\sum_{q=1}^m W(q) \leq \sum_{q=1}^m (\text{ALG} + val(i_q)) .$$

Partea stângă este valoarea totală a tuturor cadourilor:

$$\sum_{q=1}^m W(q) = \text{Val}.$$

Partea dreaptă devine:

$$\sum_{q=1}^m (\text{ALG} + \text{val}(i_q)) = \sum_{q=1}^m \text{ALG} + \sum_{q=1}^m \text{val}(i_q).$$

Deci:

$$\text{Val} \leq m \text{ALG} + \sum_{q=1}^m \text{val}(i_q).$$

Din restricția problemei:

$$\text{val}(i) \leq \frac{\text{Val}}{2m}, \quad \forall i.$$

Avem  $m$  ultime cadouri, câte unul pentru fiecare copil, deci:

$$\sum_{q=1}^m \text{val}(i_q) \leq m \cdot \frac{\text{Val}}{2m} = \frac{\text{Val}}{2}.$$

Înlocuim:

$$\text{Val} \leq m \text{ALG} + \frac{\text{Val}}{2}.$$

Scădem  $\frac{\text{Val}}{2}$  din ambele părți:

$$\frac{\text{Val}}{2} \leq m \text{ALG}.$$

Împărțim la  $m$ :

$$\boxed{\text{ALG} \geq \frac{\text{Val}}{2m}}.$$

### 3.4 d) Demonstrație că algoritmul este $\frac{1}{2}$ -aproximativ

În orice împărțire a cadourilor, valoarea medie primită de un copil este:

$$\frac{\text{Val}}{m}.$$

Copilul care primește cel mai puțin nu poate primi mai mult decât media. Prin urmare, pentru orice soluție, inclusiv pentru cea optimă:

$$\text{OPT} \leq \frac{\text{Val}}{m}.$$

Din punctul c), știm că algoritmul nostru garantează:

$$\text{ALG} \geq \frac{\text{Val}}{2m}.$$

Dar din:

$$\text{OPT} \leq \frac{\text{Val}}{m}$$

rezultă:

$$\frac{\text{OPT}}{2} \leq \frac{\text{Val}}{2m}.$$

Așadar:

$$\text{ALG} \geq \frac{\text{Val}}{2m} \geq \frac{\text{OPT}}{2}.$$

Deci:

$$\text{ALG} \geq \frac{1}{2} \text{OPT}.$$

**Concluzie:** algoritmul este  $\frac{1}{2}$ -aproximativ.

### 3.5 e) Exemplu în care algoritmul nu găsește soluția optimă

Luăm:

$$m = 2$$

și 5 cadouri cu valorile, în această ordine:

$$7, 6, 5, 5, 5.$$

Valoarea totală este:

$$\text{Val} = 7 + 6 + 5 + 5 + 5 = 28.$$

Verificăm restricția:

$$\frac{\text{Val}}{2m} = \frac{28}{4} = 7.$$

Fiecare cadou are valoare cel mult 7, deci restricția este respectată:

$$\text{val}(i) \leq 7, \quad \forall i.$$

#### Soluția dată de algoritmul greedy

Inițial:

$$W(1) = 0, \quad W(2) = 0.$$

Procesăm cadourile în ordinea dată.

| Cadou | Copil ales | $W(1)$ | $W(2)$ |
|-------|------------|--------|--------|
| 7     | copilul 1  | 7      | 0      |
| 6     | copilul 2  | 7      | 6      |
| 5     | copilul 2  | 7      | 11     |
| 5     | copilul 1  | 12     | 11     |
| 5     | copilul 2  | 12     | 16     |

La final:

$$W(1) = 12, \quad W(2) = 16.$$

Deci:

$$\text{ALG} = \min\{12, 16\} = 12.$$

### Soluția optimă

O împărțire mai bună este:

$$\begin{aligned} \text{copilul 1} : 7 + 6 &= 13, \\ \text{copilul 2} : 5 + 5 + 5 &= 15. \end{aligned}$$

Deci:

$$\min\{13, 15\} = 13.$$

Arătăm că nu se poate obține mai mult de 13. Media este:

$$\frac{\text{Val}}{m} = \frac{28}{2} = 14.$$

Pentru ca minimul să fie 14, ar trebui să împărțim exact în:

$$14 \text{ și } 14.$$

Dar nu există o submulțime a cadourilor:

$$\{7, 6, 5, 5, 5\}$$

cu suma 14.

Prin urmare:

$$\text{OPT} = 13.$$

Deci algoritmul nu este optim pe această instanță:

$$\text{ALG} = 12 < 13 = \text{OPT}.$$

# Rezumat pentru examen

## 1. Vouchere

Problemă de maximizare:

$$\max \sum_{i \in L} C_i, \quad \sum_{i \in L} C_i \leq S.$$

Algoritm:

- iei greedy până când primul obiect nu mai încap;
- compari suma deja luată cu primul obiect care nu încap;
- alegi varianta mai bună.

Formula importantă:

$$A_s + C_j > S \geq \text{OPT}.$$

Concluzie:

$$\text{ALG} \geq \frac{1}{2} \text{OPT}.$$

## 2. Camioane

Problemă de minimizare: minimizăm numărul de camioane.

Algoritm: Next Fit.

Ideea-cheie:

$$L_i + L_{i+1} > G.$$

Concluzie:

$$\text{ALG} \leq 2 \text{OPT}.$$

## 3. Cadouri

Problemă de maximizare:

$$\max_k \min W(k).$$

Algoritm:

- fiecare cadou se dă copilului care are suma minimă în acel moment;
- folosim min-heap;
- complexitate  $O(n \log m)$ .

Relația-cheie:

$$W(q) - \text{val}(i_q) \leq \text{ALG}.$$

Demonstrația principală:

$$\begin{aligned}\text{Val} &\leq m \text{ALG} + \frac{\text{Val}}{2} \\ \Rightarrow \text{ALG} &\geq \frac{\text{Val}}{2m}.\end{aligned}$$

Cum:

$$\text{OPT} \leq \frac{\text{Val}}{m},$$

rezultă:

$$\text{ALG} \geq \frac{1}{2} \text{OPT}.$$

## Cele mai importante formule

$$\text{Maximizare } \frac{1}{2}\text{-aprox:} \quad \text{ALG} \geq \frac{1}{2} \text{OPT}.$$

$$\text{Minimizare } 2\text{-aprox:} \quad \text{ALG} \leq 2 \text{OPT}.$$

$$\text{Vouchere:} \quad A_s + C_j > S \geq \text{OPT}.$$

$$\text{Camioane:} \quad L_i + L_{i+1} > G.$$

$$\text{Cadouri:} \quad W(q) - \text{val}(i_q) \leq \text{ALG}.$$

$$\text{Cadouri lower bound:} \quad \text{ALG} \geq \frac{\text{Val}}{2m}.$$

$$\text{Cadouri upper bound optim:} \quad \text{OPT} \leq \frac{\text{Val}}{m}.$$

## **Anexa B - Seminarul 3: algoritmi genetici**

# Seminar 3 – Algoritmi Genetici

## Rezolvare detaliată pentru examen

Material de lucru pentru Overleaf

### Cuprins



# 1 Ce trebuie să știi pentru examen

**Ideea de bază.** Un algoritm genetic este o metaheuristică pentru probleme de optimizare. Nu garantează soluția optimă, dar caută soluții bune într-un spațiu mare de soluții candidat.

## 1.1 Terminologie esențială

| Termen      | Semnificație                                                                                                   |
|-------------|----------------------------------------------------------------------------------------------------------------|
| Cromozom    | O soluție candidată, de obicei codificată ca șir binar $c = (g_1, g_2, \dots, g_\ell) \in \{0, 1\}^\ell$ .     |
| Genă        | O poziție din cromozom. Valoarea genei se numește alelă.                                                       |
| Populație   | Mulțimea de cromozomi la generația curentă, notată $P(t)$ . Dimensiunea populației este de obicei constantă.   |
| Fitness     | Funcție $f$ care măsoară calitatea unui individ. Pentru maximizare: fitness mai mare înseamnă individ mai bun. |
| Selecție    | Alegerea indivizilor care vor participa la producerea generației următoare.                                    |
| Încrucișare | Combinarea a doi părinți pentru a produce descendenți.                                                         |
| Mutație     | Schimbarea aleatorie a unor gene, pentru păstrarea diversității.                                               |
| Elitism     | Copierea celui mai bun individ direct în generația următoare.                                                  |

## 1.2 Schema generală a unui algoritm genetic

### Algoritm Genetic

1. Generează populația inițială  $P(0)$ , cu  $n$  cromozomi.
2. Pentru fiecare generație  $t$ :
  - a) Calculează fitness-ul fiecărui individ.
  - b) Aplică selecția:  $P(t) \rightarrow P_1(t)$ .
  - c) Aplică încrucișarea cu probabilitate  $p_c$ :  $P_1(t) \rightarrow P_2(t)$ .
  - d) Aplică mutația cu probabilitate  $p_m$ :  $P_2(t) \rightarrow P(t+1)$ .
  - e) Optional, aplică elitism.
3. Returnează cel mai bun individ găsit.

## 1.3 Codificare binară pe un interval $[a, b]$

Avem un domeniu real:

$$D = [a, b]$$

și vrem precizie de  $p$  zecimale. Numărul de subintervale cerut este:

$$N = (b - a) \cdot 10^p.$$

Un cromozom cu  $\ell$  biți poate codifica:

$$2^\ell$$

valori distincte. De aceea alegem lungimea minimă:

$$\ell = \lceil \log_2 ((b - a) \cdot 10^p) \rceil.$$

**De reținut:** folosim logaritm în baza 2 pentru că fiecare bit are două valori posibile, iar  $\ell$  biți dau  $2^\ell$  combinații.

## 1.4 Decodificare: cromozom binar $\rightarrow$ valoare reală

Pentru cromozomul:

$$c = (g_1, g_2, \dots, g_\ell), \quad g_i \in \{0, 1\},$$

mai întâi îl transformăm în număr întreg:

$$v = \sum_{k=1}^{\ell} g_k \cdot 2^{\ell-k}.$$

Apoi îl translatăm liniar în intervalul  $[a, b]$ :

$$x = a + v \cdot \frac{b - a}{2^\ell - 1}.$$

## 1.5 Codificare: valoare reală $\rightarrow$ cromozom binar

Dacă avem o valoare reală  $x \in [a, b]$ , calculăm întâi indicele întreg:

$$v = \text{round} \left( (x - a) \cdot \frac{2^\ell - 1}{b - a} \right).$$

Apoi scriem  $v$  în baza 2 pe exact  $\ell$  biți.

## 1.6 Selecția proporțională: metoda ruletei

Pentru populația:

$$P(t) = \{X_1, X_2, \dots, X_n\},$$

cu fitness-uri  $f(X_i)$ , probabilitatea de selecție a individului  $X_i$  este:

$$p_i = \frac{f(X_i)}{\sum_{j=1}^n f(X_j)}.$$

Probabilitățile cumulative sunt:

$$q_i = \sum_{j=1}^i p_j, \quad q_0 = 0.$$

Dacă generăm  $u \in [0, 1)$ , selectăm individul  $X_i$  pentru care:

$$q_{i-1} \leq u < q_i.$$

## 1.7 Încrucișare

Pentru doi cromozomi părinți  $A$  și  $B$ :

- la **un punct de tăiere**  $k$ , primul descendent ia primele  $k$  gene din  $A$  și restul din  $B$ ;
- al doilea descendent ia primele  $k$  gene din  $B$  și restul din  $A$ ;
- la **două puncte de tăiere**, se interschimbează segmentul dintre cele două puncte;
- la **încrucișare uniformă**, o mască decide de la ce părinte se ia fiecare genă.

## 1.8 Mutație

La mutația per genă, fiecare genă se modifică independent cu probabilitate  $p_m$ :

$$g_j \leftarrow 1 - g_j.$$

Numărul așteptat de mutații într-un cromozom de lungime  $\ell$  este:

$$\mathbb{E}[\text{mutații}] = \ell \cdot p_m.$$

## 2 Exercițiul 1 – Lungimea cromozomului

### Enunț

Fie domeniul:

$$D = [1, 3]$$

și precizia:

$$p = 4.$$

Se cere:

- a) numărul de subintervale;
- b) lungimea minimă a cromozomului;
- c) numărul de poziții distincte codificate și comparația rezoluției.

### Rezolvare

#### a) Numărul de subintervale

Formula este:

$$N = (b - a) \cdot 10^p.$$

În cazul nostru:

$$a = 1, \quad b = 3, \quad p = 4.$$

Deci:

$$N = (3 - 1) \cdot 10^4 = 2 \cdot 10000 = 20000.$$

$$\boxed{N = 20000}$$

#### b) Lungimea minimă a cromozomului

Căutăm cel mai mic  $\ell$  astfel încât:

$$2^\ell \geq 20000.$$

Prin formula generală:

$$\ell = \lceil \log_2(20000) \rceil.$$

Calculăm aproximativ:

$$\log_2(20000) \approx 14.2877.$$

Prin urmare:

$$\ell = \lceil 14.2877 \rceil = 15.$$

$$\boxed{\ell = 15 \text{ biți}}$$

**c) Poziții distincte și rezoluție efectivă**

Cu 15 biți putem reprezenta:

$$2^{15} = 32768$$

poziții distincte.

Rezoluția efectivă, adică pasul dintre două valori consecutive decodificate, este:

$$\Delta = \frac{b - a}{2^\ell - 1} = \frac{3 - 1}{2^{15} - 1} = \frac{2}{32767}.$$

Numeric:

$$\Delta \approx 0.000061 = 6.1 \cdot 10^{-5}.$$

Precizia cerută era  $10^{-4} = 0.0001$ . Observăm că:

$$6.1 \cdot 10^{-5} < 10^{-4}.$$

Deci rezoluția efectivă este mai bună decât cea cerută.

|                                                                        |
|------------------------------------------------------------------------|
| $2^{15} = 32768$ poziții distincte, rezoluție mai fină decât $10^{-4}$ |
|------------------------------------------------------------------------|

### 3 Exercițiul 2 – Decodificarea unui cromozom

#### Enunț

Fie:

$$D = [0, 1], \quad p = 2,$$

și cromozomul:

$$c = (1, 0, 1, 1, 0, 0, 1).$$

#### Rezolvare

##### a) Verificarea lungimii $\ell = 7$

Pentru  $D = [0, 1]$  și  $p = 2$ :

$$N = (b - a) \cdot 10^p = (1 - 0) \cdot 10^2 = 100.$$

Trebuie să avem:

$$2^\ell \geq 100.$$

Pentru  $\ell = 7$ :

$$2^7 = 128 \geq 100.$$

Deci  $\ell = 7$  este suficient.

$\ell = 7$  este suficient

##### b) Valoarea întreagă $v$

Cromozomul este:

$$c = (1, 0, 1, 1, 0, 0, 1).$$

Valoarea întreagă se calculează prin:

$$v = \sum_{k=1}^7 g_k \cdot 2^{7-k}.$$

Aplicăm formula:

$$v = 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0.$$

$$v = 1 \cdot 64 + 0 \cdot 32 + 1 \cdot 16 + 1 \cdot 8 + 0 \cdot 4 + 0 \cdot 2 + 1 \cdot 1.$$

$$v = 64 + 16 + 8 + 1 = 89.$$

$v = 89$

**c) Decodificarea în valoare reală**

Folosim formula:

$$x = a + v \cdot \frac{b - a}{2^\ell - 1}.$$

Aici:

$$a = 0, \quad b = 1, \quad v = 89, \quad \ell = 7.$$

Deci:

$$x = 0 + 89 \cdot \frac{1 - 0}{2^7 - 1} = 89 \cdot \frac{1}{127} = \frac{89}{127}.$$

$$x \approx 0.7008.$$

$$\boxed{x \approx 0.7008}$$

## 4 Exercițiul 3 – Codificarea unui număr real

### Enunț

Fie:

$$D = [-2, 2], \quad p = 3.$$

Se cere:

- a) determinarea lui  $\ell$ ;
- b) codificarea lui  $x = 0.5$ ;
- c) decodificarea cromozomului  $(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1)$ .

### Rezolvare

#### a) Lungimea cromozomului

Numărul de subintervale este:

$$N = (b - a) \cdot 10^p.$$

Aici:

$$a = -2, \quad b = 2, \quad p = 3.$$

Deci:

$$N = (2 - (-2)) \cdot 10^3 = 4 \cdot 1000 = 4000.$$

Căutăm:

$$\ell = \lceil \log_2(4000) \rceil.$$

Cum:

$$\log_2(4000) \approx 11.9658,$$

rezultă:

$$\ell = 12.$$

$\ell = 12 \text{ biți}$

#### b) Codificarea valorii $x = 0.5$

Folosim formula inversă:

$$v = \text{round} \left( (x - a) \cdot \frac{2^\ell - 1}{b - a} \right).$$

Înlocuim:

$$x = 0.5, \quad a = -2, \quad b = 2, \quad \ell = 12.$$

$$v = \text{round} \left( (0.5 - (-2)) \cdot \frac{2^{12} - 1}{2 - (-2)} \right).$$

$$v = \text{round} \left( 2.5 \cdot \frac{4095}{4} \right).$$

$$\frac{4095}{4} = 1023.75.$$



$$v = \text{round}(2.5 \cdot 1023.75) = \text{round}(2559.375) = 2559.$$

Acum transformăm 2559 în binar pe 12 biți:

$$2559 = (100111111111)_2.$$

Deci:

$$x = 0.5 \Rightarrow c = (1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1)$$

### c) Decodificarea cromozomului dat

Cromozomul are 12 biți:

$$c = (0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1).$$

Valoarea întreagă este:

$$v = 1.$$

Folosim formula:

$$x = a + v \cdot \frac{b - a}{2^\ell - 1}.$$

$$x = -2 + 1 \cdot \frac{2 - (-2)}{2^{12} - 1} = -2 + \frac{4}{4095}.$$

$$x \approx -2 + 0.0009768 = -1.9990232.$$

$$x \approx -1.9990$$

## 5 Exercițiul 4 – Selecția proporțională

### Enunț

Avem  $n = 5$  indivizi cu fitness-urile:

$$f(X_1) = 2.0, \quad f(X_2) = 5.0, \quad f(X_3) = 1.5, \quad f(X_4) = 3.5, \quad f(X_5) = 3.0.$$

Numerele aleatoare sunt:

$$u = (0.14, 0.62, 0.31, 0.88, 0.05).$$

### Rezolvare

#### a) Probabilitățile de selecție

Fitness total:

$$F = 2.0 + 5.0 + 1.5 + 3.5 + 3.0 = 15.0.$$

Formula este:

$$p_i = \frac{f(X_i)}{F}.$$

| Individ | Fitness | Probabilitate $p_i$       |
|---------|---------|---------------------------|
| $X_1$   | 2.0     | $\frac{2}{15} = 0.1333$   |
| $X_2$   | 5.0     | $\frac{5}{15} = 0.3333$   |
| $X_3$   | 1.5     | $\frac{1.5}{15} = 0.1000$ |
| $X_4$   | 3.5     | $\frac{3.5}{15} = 0.2333$ |
| $X_5$   | 3.0     | $\frac{3}{15} = 0.2000$   |

#### b) Probabilitățile cumulative

$$q_i = p_1 + p_2 + \dots + p_i.$$

| Individ | Interval ruletă    | Cumulativ $q_i$ |
|---------|--------------------|-----------------|
| $X_1$   | $[0, 0.1333)$      | 0.1333          |
| $X_2$   | $[0.1333, 0.4666)$ | 0.4666          |
| $X_3$   | $[0.4666, 0.5666)$ | 0.5666          |
| $X_4$   | $[0.5666, 0.7999)$ | 0.7999          |
| $X_5$   | $[0.7999, 1)$      | 1.0000          |

#### c) Selecțiile prin metoda ruletei

Pentru fiecare  $u$ , alegem individul al cărui interval îl conține.

| $u$  | Interval           | Individ selectat |
|------|--------------------|------------------|
| 0.14 | $[0.1333, 0.4666)$ | $X_2$            |
| 0.62 | $[0.5666, 0.7999)$ | $X_4$            |
| 0.31 | $[0.1333, 0.4666)$ | $X_2$            |
| 0.88 | $[0.7999, 1)$      | $X_5$            |
| 0.05 | $[0, 0.1333)$      | $X_1$            |

Deci populația selectată este:

$$P_1 = \{X_2, X_4, X_2, X_5, X_1\}.$$

d) Numărul de copii

| Individ | Număr de apariții în populația selectată |
|---------|------------------------------------------|
| $X_1$   | 1                                        |
| $X_2$   | 2                                        |
| $X_3$   | 0                                        |
| $X_4$   | 1                                        |
| $X_5$   | 1                                        |

**Observație pentru examen.** Individul  $X_2$  are fitness-ul cel mai mare și apare de două ori. Individul  $X_3$  are fitness-ul cel mai mic și nu apare deloc. Asta arată că selecția proporțională favorizează indivizii mai buni, dar păstrează și o parte din diversitate.

## 6 Exercițiul 5 – Selecție turneu

### Enunț

Folosim aceiași indivizi:

$$f(X_1) = 2.0, \quad f(X_2) = 5.0, \quad f(X_3) = 1.5, \quad f(X_4) = 3.5, \quad f(X_5) = 3.0.$$

Turneele de dimensiune  $k = 2$  sunt:

$$(X_3, X_5), (X_1, X_2), (X_4, X_3), (X_2, X_5), (X_1, X_4).$$

### Rezolvare

La selecția turneu alegem cel mai bun individ din fiecare pereche.

| Turneu       | Comparație fitness | Câștigător |
|--------------|--------------------|------------|
| $(X_3, X_5)$ | $1.5 < 3.0$        | $X_5$      |
| $(X_1, X_2)$ | $2.0 < 5.0$        | $X_2$      |
| $(X_4, X_3)$ | $3.5 > 1.5$        | $X_4$      |
| $(X_2, X_5)$ | $5.0 > 3.0$        | $X_2$      |
| $(X_1, X_4)$ | $2.0 < 3.5$        | $X_4$      |

Populația obținută este:

$$P_1 = \{X_5, X_2, X_4, X_2, X_4\}.$$

### Comparație cu ruleta

Prin metoda ruletei am obținut:

$$\{X_2, X_4, X_2, X_5, X_1\}.$$

Prin turneu am obținut:

$$\{X_5, X_2, X_4, X_2, X_4\}.$$

**Concluzie.** Turneul are o presiune de selecție mai mare. Individul slab  $X_3$  dispăre, iar  $X_1$  dispăre și el. Indivizii buni, mai ales  $X_2$  și  $X_4$ , sunt favorizați mai puternic.

## 7 Exercițiul 6 – Încrucișare

### Enunț

Avem doi cromozomi de lungime  $\ell = 8$ :

$$A = (1, 1, 0, 0, 1, 0, 1, 1),$$

$$B = (0, 1, 1, 1, 0, 0, 0, 1).$$

### a) Încrucișare cu un punct, $k = 3$

Punctul de tăiere este după primele 3 gene:

$$A = (1, 1, 0 \mid 0, 1, 0, 1, 1),$$

$$B = (0, 1, 1 \mid 1, 0, 0, 0, 1).$$

Descendenții sunt:

$$D_1 = (\text{primele 3 din } A \mid \text{restul din } B) = (1, 1, 0 \mid 1, 0, 0, 0, 1).$$

$$D_2 = (\text{primele 3 din } B \mid \text{restul din } A) = (0, 1, 1 \mid 0, 1, 0, 1, 1).$$

$$\boxed{D_1 = (1, 1, 0, 1, 0, 0, 0, 1)}$$

$$\boxed{D_2 = (0, 1, 1, 0, 1, 0, 1, 1)}$$

### b) Încrucișare cu două puncte, $k_1 = 2, k_2 = 6$

Împărțim cromozomii în trei segmente:

$$A = (1, 1 \mid 0, 0, 1, 0 \mid 1, 1),$$

$$B = (0, 1 \mid 1, 1, 0, 0 \mid 0, 1).$$

La încrucișare cu două puncte, segmentul din mijloc se ia de la celălalt părinte.

$$D_1 = (1, 1 \mid 1, 1, 0, 0 \mid 1, 1),$$

$$D_2 = (0, 1 \mid 0, 0, 1, 0 \mid 0, 1).$$

$$\boxed{D_1 = (1, 1, 1, 1, 0, 0, 1, 1)}$$

$$\boxed{D_2 = (0, 1, 0, 0, 1, 0, 0, 1)}$$

### c) Încrucișare uniformă

Masca este:

$$M = (1, 0, 1, 1, 0, 1, 0, 1).$$

Regula:

- dacă  $M_i = 1$ , atunci  $D_1$  ia gena de la  $A$ ;
- dacă  $M_i = 0$ , atunci  $D_1$  ia gena de la  $B$ ;
- $D_2$  ia complementar.

| Poziție | $A_i$ | $B_i$ | $M_i$ | $D_{1,i}$ |
|---------|-------|-------|-------|-----------|
| 1       | 1     | 0     | 1     | 1         |
| 2       | 1     | 1     | 0     | 1         |
| 3       | 0     | 1     | 1     | 0         |
| 4       | 0     | 1     | 1     | 0         |
| 5       | 1     | 0     | 0     | 0         |
| 6       | 0     | 0     | 1     | 0         |
| 7       | 1     | 0     | 0     | 0         |
| 8       | 1     | 1     | 1     | 1         |

Deci:

$$D_1 = (1, 1, 0, 0, 0, 0, 0, 1).$$

Complementar:

$$D_2 = (0, 1, 1, 1, 1, 0, 1, 1).$$

$$D_1 = (1, 1, 0, 0, 0, 0, 0, 1)$$

$$D_2 = (0, 1, 1, 1, 1, 0, 1, 1)$$

## 8 Exercițiul 7 – Mutație per genă

### Enunț

Cromozomul este:

$$X = (1, 0, 0, 1, 1, 0, 1, 0),$$

cu lungime:

$$\ell = 8,$$

și probabilitate de mutație:

$$p_m = 0.1.$$

Valorile aleatoare sunt:

$$u = (0.03, 0.72, 0.15, 0.91, 0.08, 0.44, 0.02, 0.67).$$

### a) Aplicarea mutației

Regula pentru mutație per genă este:

$$\text{dacă } u_i < p_m, \text{ atunci } g_i \leftarrow 1 - g_i.$$

Aici  $p_m = 0.1$ , deci mutăm genele pentru care:

$$u_i < 0.1.$$

| Poziție | Genă inițială | $u_i$ | Genă finală |
|---------|---------------|-------|-------------|
| 1       | 1             | 0.03  | 0           |
| 2       | 0             | 0.72  | 0           |
| 3       | 0             | 0.15  | 0           |
| 4       | 1             | 0.91  | 1           |
| 5       | 1             | 0.08  | 0           |
| 6       | 0             | 0.44  | 0           |
| 7       | 1             | 0.02  | 0           |
| 8       | 0             | 0.67  | 0           |

Atenție: la poziția 3,  $0.15 > 0.1$ , deci **nu** se mută.

Cromozomul rezultat este:

$$X' = (0, 0, 0, 1, 0, 0, 0, 0).$$

**Observație.** Unele soluții pot considera mutată poziția 3 doar dacă ar fi avut condiția  $u_i < 0.2$  sau alt prag. Pentru pragul din enunț,  $p_m = 0.1$ , mutațiile sunt doar la pozițiile 1, 5, 7.

**b) Numărul așteptat de mutații pentru  $p_m = 1/\ell$**

Formula este:

$$\mathbb{E}[\text{mutații per cromozom}] = \ell \cdot p_m.$$

Dacă:

$$p_m = \frac{1}{\ell},$$

atunci:

$$\mathbb{E} = \ell \cdot \frac{1}{\ell} = 1.$$

|                                               |
|-----------------------------------------------|
| $\mathbb{E} = 1 \text{ mutație per cromozom}$ |
|-----------------------------------------------|

**c) Efectul lui  $p_m$**

- Dacă  $p_m$  este prea mare, algoritmul începe să se comporte aproape aleator. Se pierde informația bună acumulată prin selecție și încrucișare.
- Dacă  $p_m$  este prea mic, populația pierde diversitate și poate ajunge rapid într-un optim local. Asta se numește convergență prematură.



## 9 Exercițiul 8 – O generație completă

### Enunț

Maximizăm funcția:

$$f(x) = x^2$$

pe domeniul:

$$D = [0, 31].$$

Codificarea se face pe  $\ell = 5$  biți, deci:

$$2^5 = 32$$

valori, iar  $x = v$  direct.

Populația inițială:

| Individ | Cromozom        |
|---------|-----------------|
| $X_1$   | (0, 1, 1, 0, 1) |
| $X_2$   | (1, 1, 0, 0, 0) |
| $X_3$   | (0, 1, 0, 0, 0) |
| $X_4$   | (1, 0, 0, 1, 1) |

### a) Calculul fitness-ului

Transformăm fiecare cromozom din binar în zecimal.

$$X_1 = (01101)_2 = 13, \quad f(X_1) = 13^2 = 169.$$

$$X_2 = (11000)_2 = 24, \quad f(X_2) = 24^2 = 576.$$

$$X_3 = (01000)_2 = 8, \quad f(X_3) = 8^2 = 64.$$

$$X_4 = (10011)_2 = 19, \quad f(X_4) = 19^2 = 361.$$

| Individ | Cromozom    | $x$ | $f(x) = x^2$ |
|---------|-------------|-----|--------------|
| $X_1$   | $(01101)_2$ | 13  | 169          |
| $X_2$   | $(11000)_2$ | 24  | 576          |
| $X_3$   | $(01000)_2$ | 8   | 64           |
| $X_4$   | $(10011)_2$ | 19  | 361          |

Fitness total:

$$F = 169 + 576 + 64 + 361 = 1170.$$

**b) Probabilități de selecție și cumulative**

$$p_i = \frac{f(X_i)}{1170}.$$

| Individ | $p_i$                             | $q_i$  |
|---------|-----------------------------------|--------|
| $X_1$   | $\frac{169}{1170} \approx 0.1444$ | 0.1444 |
| $X_2$   | $\frac{576}{1170} \approx 0.4923$ | 0.6367 |
| $X_3$   | $\frac{64}{1170} \approx 0.0547$  | 0.6914 |
| $X_4$   | $\frac{361}{1170} \approx 0.3085$ | 1.0000 |

**c) Selecția cu  $u = (0.40, 0.81, 0.18, 0.63)$**

Intervalele sunt aproximativ:

$$X_1 : [0, 0.1444),$$

$$X_2 : [0.1444, 0.6367),$$

$$X_3 : [0.6367, 0.6914),$$

$$X_4 : [0.6914, 1).$$

| $u$  | Individ selectat |
|------|------------------|
| 0.40 | $X_2$            |
| 0.81 | $X_4$            |
| 0.18 | $X_2$            |
| 0.63 | $X_2$            |

Deci:

$$P_1 = \{X_2, X_4, X_2, X_2\}.$$

În cromozomi:

$$P_1 = \{(11000), (10011), (11000), (11000)\}.$$

**d) Încrucișarea**

Perechile sunt:

$$(X_1^{sel}, X_2^{sel}) = ((11000), (10011)),$$

$$(X_3^{sel}, X_4^{sel}) = ((11000), (11000)).$$

**Prima pereche, punct  $k = 3$**

$$110 \mid 00,$$

$$100 \mid 11.$$

Descendenți:

$$D_1 = 110 \mid 11 = (11011),$$

$$D_2 = 100 \mid 00 = (10000).$$

**A doua pereche, punct  $k = 2$**

$$11 \mid 000,$$

$$11 \mid 000.$$

Descendenți:

$$D_3 = 11000,$$

$$D_4 = 11000.$$

După crossover:

$$\{D_1, D_2, D_3, D_4\} = \{(11011), (10000), (11000), (11000)\}.$$

### e) Mutația rară

Se spune că individul  $D_3$  suferă mutație la poziția  $k = 4$ .

$$D_3 = (1, 1, 0, 0, 0).$$

Poziția 4 este 0, deci devine 1:

$$D'_3 = (1, 1, 0, 1, 0) = (11010).$$

Populația finală este:

$$P(1) = \{(11011), (10000), (11010), (11000)\}.$$

### f) Fitness mediu pentru $P(0)$ și $P(1)$

Pentru  $P(0)$ :

$$\bar{f}_{P(0)} = \frac{169 + 576 + 64 + 361}{4} = \frac{1170}{4} = 292.5.$$

Pentru  $P(1)$ :

$$(11011)_2 = 27, \quad f = 27^2 = 729.$$

$$(10000)_2 = 16, \quad f = 16^2 = 256.$$

$$(11010)_2 = 26, \quad f = 26^2 = 676.$$

$$(11000)_2 = 24, \quad f = 24^2 = 576.$$

Media:

$$\bar{f}_{P(1)} = \frac{729 + 256 + 676 + 576}{4}.$$

$$\bar{f}_{P(1)} = \frac{2237}{4} = 559.25.$$

$$\bar{f}_{P(0)} = 292.5$$

$$\bar{f}_{P(1)} = 559.25$$

Cum:

$$559.25 > 292.5,$$

populația s-a îmbunătățit.

## 10 Exercițiul 9 – Convergență și diversitate

### a) Ce este convergența prematură?

Convergența prematură apare atunci când populația devine prea omogenă prea repede. Adică indivizii ajung să semene foarte mult între ei înainte ca algoritmul să fi explorat suficient spațiul soluțiilor.

Efectul este că algoritmul se poate bloca într-un optim local.

Cauze posibile:

- populație prea mică;
- mutație prea rară, adică  $p_m$  prea mic;
- presiune de selecție prea mare;
- elitism prea agresiv;
- turneu cu  $k$  prea mare.

### b) Rolul elitismului

Elitismul copiază cel mai bun individ direct în generația următoare.

Avantaj:

cel mai bun fitness nu scade de la o generație la alta.

Cu alte cuvinte, dacă notăm cel mai bun fitness din generația  $t$  cu:

$$f_{best}(t),$$

atunci cu elitism avem:

$$f_{best}(t+1) \geq f_{best}(t).$$

Risc: dacă păstrăm prea mulți indivizi eliști, populația poate deveni rapid dominată de soluții asemănătoare, ceea ce duce la convergență prematură.

### c) De ce $p_c = 0$ și $p_m$ mic seamănă cu o căutare locală?

Dacă:

$$p_c = 0,$$

atunci nu există încrucișare. Deci indivizii nu mai combină informații între ei.

Dacă și:

$$p_m \text{ este mic},$$

atunci fiecare individ se modifică foarte puțin de la o generație la alta.

Prin urmare, fiecare cromozom face mici pași în vecinătatea lui. Asta seamănă cu mai multe căutări locale paralele, nu cu un algoritm genetic complet.

### d) Strategie de adaptare dinamică pentru $p_m$

O strategie simplă este:

- dacă fitness-ul mediu crește repede, păstrăm  $p_m$  mic;
- dacă fitness-ul stagnează mai multe generații, creștem  $p_m$  pentru a introduce diversitate;
- după ce apar îmbunătățiri, scădem din nou  $p_m$ .

Formal, putem folosi:

$$p_m(t+1) = \begin{cases} \min(p_m(t) \cdot \alpha, p_{max}), & \text{dacă fitness-ul stagnează,} \\ \max(p_m(t) \cdot \beta, p_{min}), & \text{dacă fitness-ul crește,} \end{cases}$$

unde:

$$\alpha > 1, \quad 0 < \beta < 1.$$

## 11 Exercițiul 10 – Aplicare la problema rucsacului 0/1

### Enunț

Avem capacitatea:

$$W = 10.$$

Obiectele sunt:

| Obiect $i$ | Valoare $v_i$ | Greutate $w_i$ |
|------------|---------------|----------------|
| 1          | 6             | 2              |
| 2          | 5             | 3              |
| 3          | 8             | 4              |
| 4          | 4             | 2              |
| 5          | 7             | 5              |
| 6          | 3             | 1              |

### a) Codificare binară naturală

Avem  $n = 6$  obiecte, deci cromozomul are lungime:

$$\ell = 6.$$

Un cromozom este:

$$c = (g_1, g_2, g_3, g_4, g_5, g_6), \quad g_i \in \{0, 1\}.$$

Interpretare:

$$g_i = \begin{cases} 1, & \text{obiectul } i \text{ este ales,} \\ 0, & \text{obiectul } i \text{ nu este ales.} \end{cases}$$

### b) Funcție de fitness

Pentru un cromozom  $c$ , definim greutatea totală:

$$W(c) = \sum_{i=1}^6 g_i w_i.$$

Valoarea totală:

$$V(c) = \sum_{i=1}^6 g_i v_i.$$

O funcție de fitness simplă este:

$$f(c) = \begin{cases} V(c), & \text{dacă } W(c) \leq 10, \\ 0, & \text{dacă } W(c) > 10. \end{cases}$$

**Idee.** Dacă soluția încapă în rucsac, fitness-ul este valoarea totală. Dacă nu încapă, fitness-ul este 0, adică o penalizăm complet.

### c) Evaluarea cromozomilor $c_1$ și $c_2$

Avem:

$$c_1 = (1, 1, 0, 1, 0, 1).$$

Obiectele selectate sunt:

$$\{1, 2, 4, 6\}.$$

Greutatea totală:

$$W(c_1) = w_1 + w_2 + w_4 + w_6 = 2 + 3 + 2 + 1 = 8.$$

Cum:

$$8 \leq 10,$$

soluția este fezabilă.

Valoarea totală:

$$V(c_1) = v_1 + v_2 + v_4 + v_6 = 6 + 5 + 4 + 3 = 18.$$

Deci:

$$\boxed{f(c_1) = 18}$$

Acum:

$$c_2 = (1, 1, 1, 0, 1, 0).$$

Obiectele selectate sunt:

$$\{1, 2, 3, 5\}.$$

Greutatea totală:

$$W(c_2) = w_1 + w_2 + w_3 + w_5 = 2 + 3 + 4 + 5 = 14.$$

Cum:

$$14 > 10,$$

soluția este infeasibilă.

Deci:

$$\boxed{f(c_2) = 0}$$

### d) Încrucișare cu punct $k = 3$

Avem:

$$c_1 = (1, 1, 0 \mid 1, 0, 1),$$

$$c_2 = (1, 1, 1 \mid 0, 1, 0).$$

Descendenții sunt:

$$D_1 = (1, 1, 0 \mid 0, 1, 0) = (1, 1, 0, 0, 1, 0),$$

$$D_2 = (1, 1, 1 \mid 1, 0, 1) = (1, 1, 1, 1, 0, 1).$$

### Evaluarea lui $D_1$

$$D_1 = (1, 1, 0, 0, 1, 0).$$

Obiecte selectate:

$$\{1, 2, 5\}.$$

Greutate:

$$W(D_1) = 2 + 3 + 5 = 10.$$

Valoare:

$$V(D_1) = 6 + 5 + 7 = 18.$$

Deci:

$$\boxed{f(D_1) = 18}$$

### Evaluarea lui $D_2$

$$D_2 = (1, 1, 1, 1, 0, 1).$$

Obiecte selectate:

$$\{1, 2, 3, 4, 6\}.$$

Greutate:

$$W(D_2) = 2 + 3 + 4 + 2 + 1 = 12.$$

Cum:

$$12 > 10,$$

soluția este infeasibilă, deci:

$$\boxed{f(D_2) = 0}$$

**Observație importantă.** Încrucișarea nu garantează că descendenții sunt fezabili. De aceea, la probleme cu restricții, fitness-ul trebuie să penalizeze soluțiile care încalcă restricțiile.



## 12 Problemă suplimentară – VCP cu algoritm genetic

### Enunț informal

Avem un graf cu 10 vârfuri și vrem să aproximăm problema Vertex Cover. O acoperire de vârfuri este o mulțime  $S \subseteq V$  astfel încât fiecare muchie are cel puțin un capăt în  $S$ .

### Codificare

Pentru un graf cu 10 vârfuri, cromozomul are lungime:

$$\ell = 10.$$

$$c = (g_1, g_2, \dots, g_{10}), \quad g_i \in \{0, 1\}.$$

Interpretare:

$$g_i = \begin{cases} 1, & \text{vârful } i \text{ este inclus în acoperire,} \\ 0, & \text{vârful } i \text{ nu este inclus în acoperire.} \end{cases}$$

### Funcție de fitness

Problema VCP este o problemă de minimizare: vrem cât mai puține vârfuri, dar trebuie acoperite toate muchiile.

Pentru un cromozom  $c$ , definim:

$$S(c) = \{i \mid g_i = 1\}.$$

Numărul de vârfuri alese:

$$|S(c)| = \sum_{i=1}^{10} g_i.$$

Numărul de muchii neacoperite:

$$U(c) = |\{(u, v) \in E \mid g_u = 0 \text{ și } g_v = 0\}|.$$

O funcție de fitness pentru maximizare poate fi:

$$f(c) = \frac{1}{1 + |S(c)| + M \cdot U(c)},$$

unde  $M$  este o constantă mare, de exemplu:

$$M = 100.$$

**Interpretare.** Penalizăm puternic muchiile neacoperite. Dintre soluțiile valide, vor fi preferate cele cu mai puține vârfuri.

## Comparație cu ApproxVertexCover

Algoritmul clasic ApproxVertexCover:

```
S = multimea vida
E' = E
cat timp E' nu este vida:
    alege o muchie (u,v) din E'
    adauga u si v in S
    sterge toate muchiile incidente cu u sau v
return S
```

Acest algoritm are garanție teoretică:

$$|S_{alg}| \leq 2 \cdot |S_{opt}|.$$

Un algoritm genetic nu are, în general, o garanție teoretică de tip 2-aproximativ, dar poate găsi soluții bune practic, mai ales dacă rulăm suficient de multe generații.

## 13 Rezumat foarte scurt pentru examen

### Formule de memorat

$$\ell = \lceil \log_2((b - a) \cdot 10^p) \rceil$$

$$v = \sum_{k=1}^{\ell} g_k 2^{\ell-k}$$

$$x = a + v \cdot \frac{b - a}{2^{\ell} - 1}$$

$$v = \text{round} \left( (x - a) \cdot \frac{2^{\ell} - 1}{b - a} \right)$$

$$p_i = \frac{f(X_i)}{\sum_j f(X_j)}$$

$$q_i = p_1 + \dots + p_i$$

$$\mathbb{E}[\text{mutații}] = \ell p_m$$

### Idei-cheie

- La codificare,  $\ell$  trebuie să fie suficient ca  $2^{\ell}$  să acopere numărul de valori cerut.
- La decodificare, întâi transformi binar  $\rightarrow$  zecimal, apoi aplici translația liniară.
- La ruletă, indivizii cu fitness mai mare au intervale mai mari.
- La turneu, alegi cel mai bun dintr-un grup mic; presiunea de selecție e mai mare.
- La crossover, descendenții pot fi mai buni, mai slabi sau infeasibili.
- La mutație, genele se inversează:  $0 \leftrightarrow 1$ .
- La knapsack, cromozomul spune ce obiecte alegi, iar fitness-ul trebuie să penalizeze depășirea capacității.
- Elitismul păstrează cel mai bun individ, dar poate grăbi convergența prematură.

## Anexa C - Seminarul 5: geometrie computatională

# Rezolvare detaliata - Seminar 5

## Geometrie computationala

Material pentru examen

### Cuprins

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Ce trebuie retinut pentru examen                                           | 2  |
| 1 Exerciitiul 1 - Coliniaritate si rapoarte in $\mathbb{R}^3$              | 3  |
| 2 Exerciitiul 2 - Testul de orientare                                      | 6  |
| 3 Exerciitiul 3 - Graham scan, varianta Andrew                             | 8  |
| 4 Exerciitiul 4 - Exemplu cu maxim 6 elemente in lista si final 4 elemente | 10 |
| 5 Exerciitiul 5 - Acoperire convexa prin Divide et impera                  | 12 |
| Rezumat final rapid                                                        | 14 |

## Ce trebuie reținut pentru examen

- Pentru coliniaritate în  $\mathbb{R}^3$ , verificăm dacă vectorii directori sunt proporționali.
- Raportul  $r(A, B, C)$  este scalarul  $r$  pentru care

$$\overrightarrow{AB} = r\overrightarrow{BC}.$$

- Pentru orientare în plan folosim determinantul:

$$\Delta(P, Q, R) = \begin{vmatrix} 1 & 1 & 1 \\ x_P & x_Q & x_R \\ y_P & y_Q & y_R \end{vmatrix} = (x_Q - x_P)(y_R - y_P) - (y_Q - y_P)(x_R - x_P).$$

- Interpretarea semnului:

$$\Delta(P, Q, R) > 0 \Rightarrow R \text{ este la stanga muchiei orientate } \overrightarrow{PQ},$$

$$\Delta(P, Q, R) < 0 \Rightarrow R \text{ este la dreapta muchiei orientate } \overrightarrow{PQ},$$

$$\Delta(P, Q, R) = 0 \Rightarrow P, Q, R \text{ sunt coliniare.}$$

- Pentru Graham scan - varianta Andrew, sortăm punctele după coordonata  $x$  și construim marginea inferioară. Dacă ultimele trei puncte nu formează viraj la stanga, eliminăm punctul din mijloc.
- Regula folosită în aceste exerciții este:

$$\text{cat timp } \Delta(A, B, C) \leq 0, \text{ eliminăm } B.$$

Această regulă elimină și punctele coliniare intermediare.

- Algoritmul divide et impera pentru acoperirea convexă are complexitatea:

$$T(n) = 2T(n/2) + O(n) = O(n \log n).$$

## 1. Exercițiul 1 - Coliniaritate și rapoarte în $\mathbb{R}^3$

### Enunț

Fie punctele

$$A = (1, 2, 3), \quad B = (4, 5, 6) \in \mathbb{R}^3.$$

- Fie  $C = (a, 7, 8)$ . Aratați că există  $a$  astfel încât punctele  $A, B, C$  să fie coliniare și, pentru acest  $a$ , calculați raportul  $r(A, B, C)$ .
- Determinați punctul  $P$  astfel încât  $r(A, P, B) = 1$ .
- Dati un exemplu de punct  $Q$  astfel încât  $r(A, B, Q) < 0$  și  $r(A, Q, B) < 0$ .

### Notiune importantă: raportul $r(A, B, C)$

Dacă punctele  $A, B, C$  sunt coliniare, raportul

$$r(A, B, C)$$

este scalarul  $r$  pentru care

$$\overrightarrow{AB} = r\overrightarrow{BC}.$$

Deci trebuie să comparăm vectorii  $\overrightarrow{AB}$  și  $\overrightarrow{BC}$ .

### Rezolvare a)

Calculăm vectorii:

$$\overrightarrow{AB} = B - A = (4 - 1, 5 - 2, 6 - 3) = (3, 3, 3).$$

Pentru  $C = (a, 7, 8)$ :

$$\overrightarrow{BC} = C - B = (a - 4, 7 - 5, 8 - 6) = (a - 4, 2, 2).$$

Punctele  $A, B, C$  sunt coliniare dacă vectorii  $\overrightarrow{AB}$  și  $\overrightarrow{BC}$  sunt proporționali. Cum

$$\overrightarrow{AB} = (3, 3, 3),$$

vectorul  $\overrightarrow{BC}$  trebuie să aibă toate coordonatele egale, deoarece este proporțional cu  $(1, 1, 1)$ . Avem deja ultimele două coordonate egale cu 2, deci trebuie să avem:

$$a - 4 = 2.$$

Rezulta:

$$\boxed{a = 6}.$$

Pentru  $a = 6$ :

$$C = (6, 7, 8),$$

$$\overrightarrow{BC} = (6 - 4, 7 - 5, 8 - 6) = (2, 2, 2).$$

Acum comparăm:

$$\overrightarrow{AB} = (3, 3, 3), \quad \overrightarrow{BC} = (2, 2, 2).$$

Deci:

$$(3, 3, 3) = \frac{3}{2}(2, 2, 2).$$

Prin definiție:

$$\overrightarrow{AB} = r(A, B, C)\overrightarrow{BC},$$

asa ca:

$$\boxed{r(A, B, C) = \frac{3}{2}}.$$

**Rezolvare b)**

Conditia este:

$$r(A, P, B) = 1.$$

Prin definitia raportului:

$$\overrightarrow{AP} = 1 \cdot \overrightarrow{PB},$$

adica:

$$\overrightarrow{AP} = \overrightarrow{PB}.$$

Aceasta inseamna ca  $P$  este mijlocul segmentului  $[AB]$ . Formula mijlocului segmentului este:

$$P = \frac{A + B}{2}.$$

Aplicam coordonata cu coordonata:

$$P = \left( \frac{1+4}{2}, \frac{2+5}{2}, \frac{3+6}{2} \right).$$

Deci:

$$P = \left( \frac{5}{2}, \frac{7}{2}, \frac{9}{2} \right).$$

**Rezolvare c)**

Vrem un punct  $Q$  astfel incat:

$$r(A, B, Q) < 0$$

si

$$r(A, Q, B) < 0.$$

Un exemplu simplu este sa luam  $Q$  pe aceeasi dreapta cu  $A$  si  $B$ , dar in partea opusa fata de  $B$ , astfel incat  $A$  sa fie intre  $Q$  si  $B$ .

Dreapta  $AB$  are directia:

$$(1, 1, 1).$$

Alegem:

$$Q = A - (1, 1, 1) = (1, 2, 3) - (1, 1, 1) = (0, 1, 2).$$

Verificam rapoartele. Pentru  $r(A, B, Q)$ :

$$\overrightarrow{AB} = (3, 3, 3),$$

$$\overrightarrow{BQ} = Q - B = (0 - 4, 1 - 5, 2 - 6) = (-4, -4, -4).$$

Cautam  $r$  astfel incat:

$$\overrightarrow{AB} = r \overrightarrow{BQ}.$$

Deci:

$$(3, 3, 3) = r(-4, -4, -4).$$

Rezulta:

$$r = -\frac{3}{4}.$$

Astfel:

$$r(A, B, Q) = -\frac{3}{4} < 0.$$

Pentru  $r(A, Q, B)$ :

$$\overrightarrow{AQ} = Q - A = (0 - 1, 1 - 2, 2 - 3) = (-1, -1, -1),$$



$$\overrightarrow{QB} = B - Q = (4 - 0, 5 - 1, 6 - 2) = (4, 4, 4).$$

Cautăm  $r$  astfel încât:

$$\overrightarrow{AQ} = r\overrightarrow{QB}.$$

Deci:

$$(-1, -1, -1) = r(4, 4, 4),$$

prin urmare:

$$r = -\frac{1}{4}.$$

Astfel:

$$r(A, Q, B) = -\frac{1}{4} < 0.$$

Un punct corect este deci:

$$Q = (0, 1, 2).$$

## 2. Exercițiul 2 - Testul de orientare

### Enunt

Fie punctele

$$P = (1, -1), \quad Q = (3, 3).$$

- Calculati valoarea determinantului care apare in testul de orientare pentru muchia orientata  $\overrightarrow{PQ}$  si punctul de testare  $O = (0, 0)$ .
- Fie  $R_\alpha = (\alpha, -\alpha)$ , unde  $\alpha \in \mathbb{R}$ . Determinati valorile lui  $\alpha$  pentru care punctul  $R_\alpha$  este situat in dreapta muchiei orientate  $\overrightarrow{PQ}$ .

### Formula testului de orientare

Pentru trei puncte

$$P = (x_P, y_P), \quad Q = (x_Q, y_Q), \quad R = (x_R, y_R),$$

folosim determinantul:

$$\Delta(P, Q, R) = \begin{vmatrix} 1 & 1 & 1 \\ x_P & x_Q & x_R \\ y_P & y_Q & y_R \end{vmatrix}.$$

O forma echivalenta, mai usor de calculat, este:

$$\Delta(P, Q, R) = (x_Q - x_P)(y_R - y_P) - (y_Q - y_P)(x_R - x_P).$$

### Rezolvare a)

Avem:

$$P = (1, -1), \quad Q = (3, 3), \quad O = (0, 0).$$

Scriem determinantul:

$$\Delta(P, Q, O) = \begin{vmatrix} 1 & 1 & 1 \\ 1 & 3 & 0 \\ -1 & 3 & 0 \end{vmatrix}.$$

Calculam prin formula cu diferente:

$$\Delta(P, Q, O) = (3 - 1)(0 - (-1)) - (3 - (-1))(0 - 1).$$

$$\Delta(P, Q, O) = 2 \cdot 1 - 4 \cdot (-1).$$

$$\Delta(P, Q, O) = 2 + 4 = 6.$$

Deci:

$$\boxed{\Delta(P, Q, O) = 6}.$$

Cum valoarea este pozitiva, rezulta ca:

$$\boxed{O \text{ este la stanga muchiei orientate } \overrightarrow{PQ}}.$$

**Rezolvare b)**

Avem:

$$R_\alpha = (\alpha, -\alpha).$$

Calculăm determinantul:

$$\Delta(P, Q, R_\alpha) = \begin{vmatrix} 1 & 1 & 1 \\ 1 & 3 & \alpha \\ -1 & 3 & -\alpha \end{vmatrix}.$$

Folosim formula cu diferite:

$$\Delta(P, Q, R_\alpha) = (3 - 1)((-\alpha) - (-1)) - (3 - (-1))(\alpha - 1).$$

$$\Delta(P, Q, R_\alpha) = 2(1 - \alpha) - 4(\alpha - 1).$$

$$\Delta(P, Q, R_\alpha) = 2 - 2\alpha - 4\alpha + 4.$$

$$\Delta(P, Q, R_\alpha) = 6 - 6\alpha = 6(1 - \alpha).$$

Punctul  $R_\alpha$  este în dreapta muchiei orientate  $\overrightarrow{PQ}$  dacă determinantul este negativ:

$$\Delta(P, Q, R_\alpha) < 0.$$

Deci:

$$6(1 - \alpha) < 0.$$

Împartim la 6 > 0:

$$1 - \alpha < 0.$$

$$\alpha > 1.$$

Prin urmare:

$$\boxed{R_\alpha \text{ este la dreapta lui } \overrightarrow{PQ} \iff \alpha > 1}.$$

### 3. Exercițiul 3 - Graham scan, varianta Andrew

#### Enunț

Fie multimea

$$M = \{P_1, P_2, \dots, P_9\},$$

unde:

$$\begin{aligned} P_1 &= (-2, 4), & P_2 &= (-1, 1), & P_3 &= (0, 1), & P_4 &= (2, 1), \\ P_5 &= (4, 3), & P_6 &= (5, 5), & P_7 &= (6, 9), & P_8 &= (8, 4), & P_9 &= (10, 6). \end{aligned}$$

Detaliați cum evoluează lista  $L_i$  a varfurilor care determina marginea inferioară a frontierei acoperirii convexe, obținută prin Graham scan - varianta Andrew.

#### Observație despre convenție

Punctele sunt deja ordonate crescător după coordonata  $x$ . Pentru marginea inferioară, adăugăm punctele în această ordine și verificăm ultimele trei puncte. Dacă ultimele trei puncte nu formează viraj la stânga, eliminăm punctul din mijloc.

Concret, pentru ultimele trei puncte  $A, B, C$  din lista:

$$\Delta(A, B, C) \leq 0 \Rightarrow \text{eliminăm } B.$$

#### Calculul evoluției listei

| Pas           | Verificare                                                                                                                                                                                                                       | Lista $L_i$               |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| Adăugăm $P_1$ | Nu avem încă 3 puncte.                                                                                                                                                                                                           | $P_1$                     |
| Adăugăm $P_2$ | Nu avem încă 3 puncte.                                                                                                                                                                                                           | $P_1 P_2$                 |
| Adăugăm $P_3$ | $\Delta(P_1, P_2, P_3) = 3 > 0$ , deci păstrăm.                                                                                                                                                                                  | $P_1 P_2 P_3$             |
| Adăugăm $P_4$ | $\Delta(P_2, P_3, P_4) = 0$ , deci eliminăm $P_3$ .<br>Apoi $\Delta(P_1, P_2, P_4) = 9 > 0$ .                                                                                                                                    | $P_1 P_2 P_4$             |
| Adăugăm $P_5$ | $\Delta(P_2, P_4, P_5) = 6 > 0$ , deci păstrăm.                                                                                                                                                                                  | $P_1 P_2 P_4 P_5$         |
| Adăugăm $P_6$ | $\Delta(P_4, P_5, P_6) = 2 > 0$ , deci păstrăm.                                                                                                                                                                                  | $P_1 P_2 P_4 P_5 P_6$     |
| Adăugăm $P_7$ | $\Delta(P_5, P_6, P_7) = 2 > 0$ , deci păstrăm.                                                                                                                                                                                  | $P_1 P_2 P_4 P_5 P_6 P_7$ |
| Adăugăm $P_8$ | $\Delta(P_6, P_7, P_8) = -13 < 0$ , eliminăm $P_7$ .<br>Apoi $\Delta(P_5, P_6, P_8) = -7 < 0$ , eliminăm $P_6$ .<br>Apoi $\Delta(P_4, P_5, P_8) = -6 < 0$ , eliminăm $P_5$ .<br>Apoi $\Delta(P_2, P_4, P_8) = 9 > 0$ , ne oprim. | $P_1 P_2 P_4 P_8$         |
| Adăugăm $P_9$ | $\Delta(P_4, P_8, P_9) = 6 > 0$ , deci păstrăm.                                                                                                                                                                                  | $P_1 P_2 P_4 P_8 P_9$     |

#### Rezultat final

Lista finală a varfurilor de pe marginea inferioară este:

$$L_i = P_1 P_2 P_4 P_8 P_9.$$

În coordonate:

$$L_i = ((-2, 4), (-1, 1), (2, 1), (8, 4), (10, 6)).$$

**De ce se elimina punctele?**

La adaugarea lui  $P_8$ , punctele  $P_7, P_6, P_5$  ajung deasupra marginii inferioare. Ele nu mai pot fi varfuri ale marginii inferioare a acoperirii convexe, deoarece produc viraj la dreapta. De aceea sunt eliminate in ordinea:

$$P_7, P_6, P_5.$$

## 4. Exercițiul 4 - Exemplu cu maxim 6 elemente in lista si final 4 elemente

### Enunt

Dati un exemplu de multime  $M$  din planul  $\mathbb{R}^2$  pentru care, la final,  $L_i$  are 4 elemente, dar, pe parcursul algoritmului, numarul maxim de elemente al lui  $L_i$  este egal cu 6.

### Exemplu propus

Luam urmatoarele 9 puncte, deja ordonate crescator dupa coordonata  $x$ :

$$\begin{aligned} P_1 &= (0, 4), & P_2 &= (1, -2), & P_3 &= (2, 2), & P_4 &= (3, -8), \\ P_5 &= (4, 10), & P_6 &= (5, 10), & P_7 &= (6, -4), & P_8 &= (7, 1), & P_9 &= (8, -8). \end{aligned}$$

### Evolutia listei

Aplicam aceeasi regula:

$$\Delta(A, B, C) \leq 0 \Rightarrow \text{eliminam punctul din mijloc } B.$$

| Pas           | Verificare                                                                                                                                                                                                                          | Lista $L_i$           |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| Adaugam $P_1$ | Nu verificam inca.                                                                                                                                                                                                                  | $P_1$                 |
| Adaugam $P_2$ | Nu verificam inca.                                                                                                                                                                                                                  | $P_1 P_2$             |
| Adaugam $P_3$ | $\Delta(P_1, P_2, P_3) = 10 > 0$ .                                                                                                                                                                                                  | $P_1 P_2 P_3$         |
| Adaugam $P_4$ | $\Delta(P_2, P_3, P_4) = -14 < 0$ , deci eliminam $P_3$ . Apoi $\Delta(P_1, P_2, P_4) = 6 > 0$ .                                                                                                                                    | $P_1 P_2 P_4$         |
| Adaugam $P_5$ | $\Delta(P_2, P_4, P_5) = 42 > 0$ .                                                                                                                                                                                                  | $P_1 P_2 P_4 P_5$     |
| Adaugam $P_6$ | Imediat dupa adaugare avem $P_1 P_2 P_4 P_5 P_6$ . Dar $\Delta(P_4, P_5, P_6) = -18 < 0$ , deci eliminam $P_5$ .                                                                                                                    | $P_1 P_2 P_4 P_6$     |
| Adaugam $P_7$ | $\Delta(P_4, P_6, P_7) = -46 < 0$ , eliminam $P_6$ . Apoi $\Delta(P_2, P_4, P_7) = 26 > 0$ .                                                                                                                                        | $P_1 P_2 P_4 P_7$     |
| Adaugam $P_8$ | $\Delta(P_4, P_7, P_8) = 11 > 0$ .                                                                                                                                                                                                  | $P_1 P_2 P_4 P_7 P_8$ |
| Adaugam $P_9$ | Imediat dupa adaugare lista are 6 elemente: $P_1 P_2 P_4 P_7 P_8 P_9$ . Apoi $\Delta(P_7, P_8, P_9) = -14 < 0$ , eliminam $P_8$ . Apoi $\Delta(P_4, P_7, P_9) = -20 < 0$ , eliminam $P_7$ . Apoi $\Delta(P_2, P_4, P_9) = 30 > 0$ . | $P_1 P_2 P_4 P_9$     |

### Concluzie

La final:

$$\boxed{L_i = P_1 P_2 P_4 P_9},$$

deci  $L_i$  are exact 4 elemente.

Pe parcurs, imediat dupa adaugarea lui  $P_9$ , lista devine:

$$P_1 P_2 P_4 P_7 P_8 P_9,$$

care are 6 elemente. După verificările de orientare,  $P_8$  și  $P_7$  sunt eliminate.

Prin urmare, exemplul respectă cerința:

$$\boxed{\max |L_i| = 6, \quad |L_i \text{ final}| = 4}.$$

## 5. Exercițiul 5 - Acoperire convexa prin Divide et impera

### Enunt

Discutati un algoritm bazat pe paradigma Divide et impera pentru determinarea acoperirii convexe. Analizati complexitatea-timp.

### Ideea problemei

Avem o multime de puncte in plan si vrem sa determinam acoperirea convexa, adica cel mai mic poligon convex care contine toate punctele.

Prin metoda Divide et impera procedam astfel:

1. Sortam punctele dupa coordonata  $x$ .
2. Impartim multimea in doua jumatati: stanga si dreapta.
3. Calculam recursiv acoperirea convexa pentru fiecare jumatate.
4. Interclasam cele doua acoperiri convexe prin gasirea tangentelor comune superioara si inferioara.

### Pseudocod

---

#### ConvexHull-DivideEtImpera( $P$ )

---

1. Sorteaza punctele din  $P$  dupa coordonata  $x$ .
  2. Daca  $|P| \leq 3$ , returneaza acoperirea convexa direct.
  3. Imparte  $P$  in doua submultimi:  $P_L$  si  $P_R$ .
  4.  $H_L \leftarrow \text{ConvexHull-DivideEtImpera}(P_L)$ .
  5.  $H_R \leftarrow \text{ConvexHull-DivideEtImpera}(P_R)$ .
  6. Determina tangenta superioara comuna a lui  $H_L$  si  $H_R$ .
  7. Determina tangenta inferioara comuna a lui  $H_L$  si  $H_R$ .
  8. Elimina muchiile interioare si concateneaza lanturile exterioare.
  9. Returneaza acoperirea convexa obtinuta.
- 

### Cum se face interclasarea

Dupa ce avem doua acoperiri convexe:

$$H_L \quad \text{si} \quad H_R,$$

tre ele exista doua tangente importante:

- tangenta comuna superioara;
- tangenta comuna inferioara.

Aceste tangente delimiteaza partea exterioara a acoperirii convexe finale. Muchiile care raman in interior intre cele doua tangente se elimina.

### Analiza complexitatii

Presupunem ca punctele sunt deja sortate dupa  $x$ . Atunci, la fiecare pas:

- rezolvam doua subprobleme de dimensiune aproximativ  $n/2$ ;
- interclasarea celor doua acoperiri se poate face in timp liniar  $O(n)$ .



Obținem recurența:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n).$$

Prin teorema Master:

$$T(n) = O(n \log n).$$

Dacă includem și sortarea inițială:

$$O(n \log n) + O(n \log n) = O(n \log n).$$

Deci complexitatea totală rămâne:

$$\boxed{O(n \log n)}.$$

### Ce trebuie scris la examen

Un răspuns complet trebuie să conțină:

- sortarea punctelor după coordonata  $x$ ;
- împartirea în două mulțimi aproximativ egale;
- calculul recursiv pentru acoperirea convexă a fiecărei jumătăți;
- combinarea prin tangenta superioară și tangenta inferioară;
- recurența  $T(n) = 2T(n/2) + O(n)$ ;
- concluzia  $T(n) = O(n \log n)$ .

## Rezumat final rapid

- **Coliniaritate in  $\mathbb{R}^3$ :** calculezi vectorii si verifici proportionalitatea.
- **Raport:**

$$r(A, B, C) = r \iff \overrightarrow{AB} = r\overrightarrow{BC}.$$

- **Orientare:**

$$\Delta(P, Q, R) = \begin{vmatrix} 1 & 1 & 1 \\ x_P & x_Q & x_R \\ y_P & y_Q & y_R \end{vmatrix}.$$

- **Semn determinant:**

$$\Delta > 0 \Rightarrow \text{stanga}, \quad \Delta < 0 \Rightarrow \text{dreapta}, \quad \Delta = 0 \Rightarrow \text{coliniar}.$$

- **Andrew lower hull:** daca ultimele trei puncte nu fac viraj la stanga, elimini punctul din mijloc.
- **Divide et impera pentru convex hull:**

$$T(n) = 2T(n/2) + O(n) = O(n \log n).$$

## **Anexa D - Seminarul 6: geometrie computationala**

Rezolvare detaliată - Seminar 6  
Algoritmi Avansați: Geometrie computațională

## Contents

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| Idei esențiale pentru examen                                                 | 2  |
| 1 Exercițiul 1 - Jarvis' march                                               | 4  |
| 2 Exercițiul 2 - Teorema Galeriei de Artă                                    | 6  |
| 3 Exercițiul 3 - Două aplicări ale Teoremei Galeriei de Artă                 | 8  |
| 3.1 Varianta I - triangulare de tip evantai . . . . .                        | 8  |
| 3.2 Varianta II - triangulare fără vârf comun tuturor diagonalelor . . . . . | 9  |
| 4 Exercițiul 4 - Poligon cu vârfuri convexe și concave, toate principale     | 10 |
| 5 Exercițiul 5 - Număr de triunghiuri și muchii într-o triangulare           | 11 |
| 6 Exercițiul 6 - Exemplu de mulțime cu 6 triunghiuri și 11 muchii            | 13 |
| 7 Exercițiul 7 - Numărul de muchii în funcție de $\alpha$                    | 14 |
| 8 Exercițiul 8 - Inegalități pentru grafuri planare                          | 16 |
| Rezumat rapid pentru examen                                                  | 19 |

# Idei esențiale pentru examen

## 1. Testul de orientare

Pentru trei puncte  $A = (x_A, y_A)$ ,  $B = (x_B, y_B)$  și  $C = (x_C, y_C)$ , testul de orientare se poate calcula prin determinant:

$$\Delta(A, B, C) = \begin{vmatrix} 1 & 1 & 1 \\ x_A & x_B & x_C \\ y_A & y_B & y_C \end{vmatrix}.$$

Echivalent, se poate folosi formula:

$$\Delta(A, B, C) = (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A).$$

Interpretare pentru muchia orientată  $\overrightarrow{AB}$ :

$$\begin{cases} \Delta(A, B, C) > 0, & C \text{ este la stânga lui } \overrightarrow{AB}, \\ \Delta(A, B, C) < 0, & C \text{ este la dreapta lui } \overrightarrow{AB}, \\ \Delta(A, B, C) = 0, & A, B, C \text{ sunt coliniare.} \end{cases}$$

## 2. Jarvis' march

Jarvis' march construiește frontiera acoperirii convexe alegând succesiv următorul punct extrem. Pentru un punct curent  $P$ , alegem inițial un pivot  $S$ , apoi testăm toate celelalte puncte. Dacă un punct  $Q$  se află mai "în exterior" decât pivotul curent, înlocuim pivotul.

În această problemă, pentru marginea inferioară parcursă în sens trigonometric, înlocuim pivotul  $S$  cu  $Q$  când:

$$\Delta(P, S, Q) < 0.$$

## 3. Teorema Galeriei de Artă

Pentru orice poligon simplu cu  $n$  vârfuri, sunt suficiente:

$$\left\lceil \frac{n}{3} \right\rceil$$

camere de supraveghere.

Metoda demonstrativă folosită la curs:

- 1) triangulăm poligonul;
- 2) colorăm vârfurile cu 3 culori astfel încât fiecare triunghi să aibă toate cele 3 culori;
- 3) alegem culoarea care apare de cele mai puține ori;
- 4) punem camere în vârfurile de acea culoare.

Deoarece fiecare triunghi are câte un vârf din fiecare culoare, fiecare triunghi este supravegheat.

#### 4. Formule pentru triangulări de mulțimi de puncte

Pentru o mulțime de  $n$  puncte din plan, dintre care  $k$  sunt pe frontiera acoperirii convexe, orice triangulare are:

$$n_t = 2n - k - 2$$

triunghiuri și

$$n_m = 3n - k - 3$$

muchii.

#### 5. Formula lui Euler pentru grafuri planare

Pentru un graf planar conex:

$$v - m + f = 2,$$

unde  $v$  este numărul de vârfuri,  $m$  numărul de muchii, iar  $f$  numărul de fețe.

De asemenea:

$$\sum_{x \in V} \deg(x) = 2m.$$

Dacă toate vârfurile au grad cel puțin 3, atunci:

$$2m \geq 3v.$$

Dacă fiecare față are grad cel puțin 3, atunci:

$$2m \geq 3f.$$

# 1 Exercițiul 1 - Jarvis' march

**Enunț.** Fie mulțimea

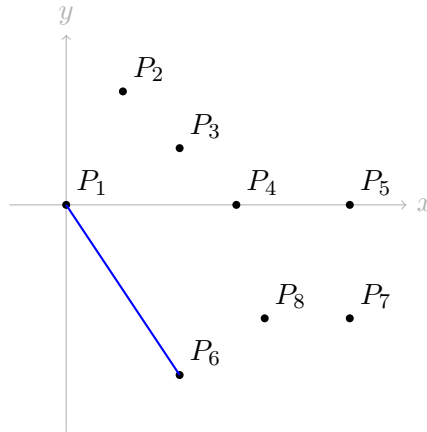
$$P = \{P_1, P_2, \dots, P_8\},$$

unde:

$$\begin{aligned} P_1 &= (0, 0), & P_2 &= (1, 2), & P_3 &= (2, 1), & P_4 &= (3, 0), \\ P_5 &= (5, 0), & P_6 &= (2, -3), & P_7 &= (5, -2), & P_8 &= (3.5, -2). \end{aligned}$$

Trebuie indicate testele necesare pentru a găsi succesorul lui  $P_1$  prin Jarvis' march, pentru marginea inferioară a acoperirii convexe, parcursă în sens trigonometric. Pivotul inițial este  $P_2$ .

**Reprezentare orientativă**



**Formula folosită**

Pentru că  $P_1 = (0, 0)$ , determinantul se simplifică. Dacă pivotul curent este  $S = (x_S, y_S)$  și testăm punctul  $Q = (x_Q, y_Q)$ , atunci:

$$\Delta(P_1, S, Q) = x_S y_Q - y_S x_Q.$$

Dacă:

$$\Delta(P_1, S, Q) < 0,$$

atunci  $Q$  este la dreapta muchiei orientate  $\overrightarrow{P_1 S}$  și devine noul pivot.

**Testele efectuate**

Inițial:

$$S = P_2 = (1, 2).$$

| Punct testat $Q$  | Pivot curent $S$ | $\Delta(P_1, S, Q)$                       | Decizie              |
|-------------------|------------------|-------------------------------------------|----------------------|
| $P_3 = (2, 1)$    | $P_2 = (1, 2)$   | $1 \cdot 1 - 2 \cdot 2 = -3 < 0$          | $P_3$ devine pivot   |
| $P_4 = (3, 0)$    | $P_3 = (2, 1)$   | $2 \cdot 0 - 1 \cdot 3 = -3 < 0$          | $P_4$ devine pivot   |
| $P_5 = (5, 0)$    | $P_4 = (3, 0)$   | $3 \cdot 0 - 0 \cdot 5 = 0$               | pivotul rămâne $P_4$ |
| $P_6 = (2, -3)$   | $P_4 = (3, 0)$   | $3 \cdot (-3) - 0 \cdot 2 = -9 < 0$       | $P_6$ devine pivot   |
| $P_7 = (5, -2)$   | $P_6 = (2, -3)$  | $2 \cdot (-2) - (-3) \cdot 5 = 11 > 0$    | pivotul rămâne $P_6$ |
| $P_8 = (3.5, -2)$ | $P_6 = (2, -3)$  | $2 \cdot (-2) - (-3) \cdot 3.5 = 6.5 > 0$ | pivotul rămâne $P_6$ |

## Concluzie

După ce au fost testate toate punctele, pivotul final este:

$$\boxed{P_6 = (2, -3)}.$$

Deci succesorul lui  $P_1$  pe marginea inferioară a acoperirii convexe este:

$$\boxed{P_6}.$$



## 2 Exercițiul 2 - Teorema Galeriei de Artă

**Enunț.** Se consideră poligonul  $P_0P_1 \dots P_{12}$ , unde:

$$\begin{aligned} P_0 &= (0, -4), & P_1 &= (5, -6), & P_2 &= (7, -4), & P_3 &= (5, -2), \\ P_4 &= (5, 2), & P_5 &= (7, 4), & P_6 &= (7, 6), \end{aligned}$$

iar punctele  $P_7, \dots, P_{12}$  sunt simetricele punctelor  $P_6, \dots, P_1$  față de axa  $Oy$ .

Prin urmare:

$$\begin{aligned} P_7 &= (-7, 6), & P_8 &= (-7, 4), & P_9 &= (-5, 2), \\ P_{10} &= (-5, -2), & P_{11} &= (-7, -4), & P_{12} &= (-5, -6). \end{aligned}$$

### Pasul 1: numărul maxim garantat de camere

Poligonul are:

$$n = 13$$

vârfuri. Prin Teorema Galeriei de Artă, sunt suficiente:

$$\left\lfloor \frac{13}{3} \right\rfloor = 4$$

camere.

### Pasul 2: o triangulare posibilă

O posibilă triangulare este dată de diagonalele:

$$\begin{aligned} &P_0P_2, P_0P_3, P_0P_4, P_{12}P_4, P_{12}P_5, P_{12}P_6, \\ &P_6P_8, P_6P_9, P_6P_{10}, P_{12}P_{10}. \end{aligned}$$

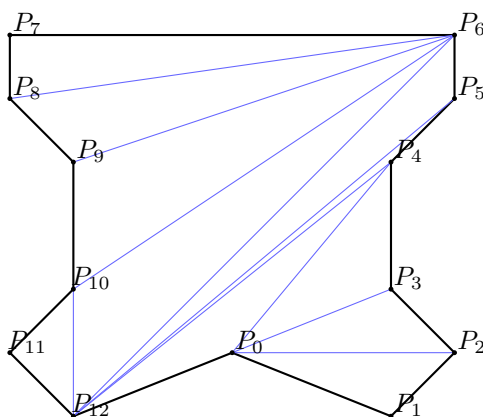
Pentru un poligon cu 13 vârfuri, o triangulare are:

$$n - 3 = 13 - 3 = 10$$

diagonale și:

$$n - 2 = 13 - 2 = 11$$

triunghiuri. Lista de mai sus are exact 10 diagonale.



### Pasul 3: o 3-colorare posibilă

Pentru această triangulare, o 3-colorare validă este:

Culoarea A:  $\{P_2, P_4, P_6, P_{11}\}$ ,

Culoarea B:  $\{P_1, P_3, P_7, P_9, P_{12}\}$ ,

Culoarea C:  $\{P_0, P_5, P_8, P_{10}\}$ .

Cardinalele sunt:

$$|A| = 4, \quad |B| = 5, \quad |C| = 4.$$

### Pasul 4: amplasarea camerelor

Putem alege o clasă de culoare cu număr minim de vârfuri. De exemplu, alegem culoarea A:

$$\boxed{\{P_2, P_4, P_6, P_{11}\}}.$$

Așadar, o posibilă amplasare a camerelor este în vârfurile:

$$\boxed{P_2, P_4, P_6, P_{11}}.$$

### De ce funcționează?

Fiecare triunghi al triangulării are câte un vârf din fiecare dintre cele 3 culori. Dacă punem camere în toate vârfurile unei culori, atunci fiecare triunghi are cel puțin o cameră într-un vârf. Cum poligonul este reuniunea triunghiurilor din triangulare, tot poligonul este supravegheat.

### 3 Exercițiul 3 - Două aplicări ale Teoremei Galeriei de Artă

**Enunț.** Fie hexagonul:

$$P = (P_1 P_2 P_3 P_4 P_5 P_6),$$

unde:

$$P_1 = (5, 0), \quad P_2 = (3, 2), \quad P_3 = (-1, 2),$$

$$P_4 = (-3, 0), \quad P_5 = (-1, -2), \quad P_6 = (3, -2).$$

Trebuie arătat că metoda din Teorema Galeriei de Artă poate da două rezultate diferite, în funcție de triangulare.

#### Observație importantă

Poligonul este convex. Pentru un hexagon, o triangulare are:

$$n - 3 = 6 - 3 = 3$$

diagonale și:

$$n - 2 = 6 - 2 = 4$$

triunghiuri.

#### 3.1 Varianta I - triangulare de tip evantai

Alegem toate diagonalele din același vârf, de exemplu din  $P_1$ :

$$P_1 P_3, \quad P_1 P_4, \quad P_1 P_5.$$

Triunghiurile sunt:

$$(P_1 P_2 P_3), (P_1 P_3 P_4), (P_1 P_4 P_5), (P_1 P_5 P_6).$$

O 3-colorare posibilă este:

Culoarea A:  $\{P_1\}$ ,

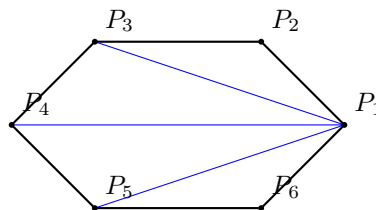
Culoarea B:  $\{P_2, P_4, P_6\}$ ,

Culoarea C:  $\{P_3, P_5\}$ .

Cea mai mică clasă are un singur vârf:

$$\boxed{\{P_1\}}.$$

Prin metoda din teoremă, este suficientă o singură cameră, amplasată în  $P_1$ .



### 3.2 Varianta II - triangulare fără vârf comun tuturor diagonalelor

Alegem diagonalele:

$$P_1P_3, \quad P_3P_5, \quad P_5P_1.$$

Triunghiurile obținute sunt:

$$(P_1P_2P_3), (P_3P_4P_5), (P_5P_6P_1), (P_1P_3P_5).$$

O 3-colorare posibilă este:

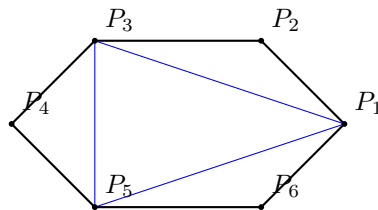
Culoarea A:  $\{P_1, P_4\}$ ,

Culoarea B:  $\{P_3, P_6\}$ ,

Culoarea C:  $\{P_5, P_2\}$ .

Fiecare clasă are câte două vârfuri, deci metoda indică două camere. De exemplu, alegem:

$$\boxed{\{P_1, P_4\}}.$$



### Concluzie

Același poligon poate conduce, prin metoda bazată pe triangulare și 3-colorare, la:

$$\boxed{1 \text{ cameră}}$$

în prima triangulare și la:

$$\boxed{2 \text{ camere}}$$

în a doua triangulare.

**Atenție pentru examen:** aici discutăm numărul de camere indicat de metoda demonstrativă a teoremei, care depinde de triangularea aleasă.

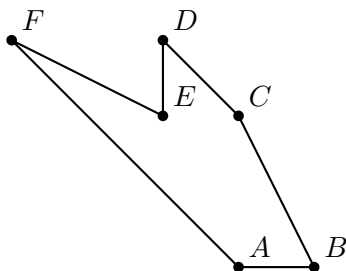
## 4 Exercițiul 4 - Poligon cu vârfuri convexe și concave, toate principale

**Cerință.** Să se dea un exemplu de poligon cu 6 vârfuri care are atât vârfuri convexe, cât și vârfuri concave, iar toate vârfurile sunt principale.

### Exemplu

Considerăm poligonul  $ABCDEF$  cu:

$$A = (3, 0), \quad B = (4, 0), \quad C = (3, 2), \quad D = (2, 3), \quad E = (2, 2), \quad F = (0, 3).$$



### Verificarea tipului de vârf

Poligonul este parcurs în sens trigonometric. Folosim semnul orientării:

$$\Delta(V_{i-1}, V_i, V_{i+1}).$$

Pentru un poligon parcurs trigonometric:

$$\Delta > 0 \Rightarrow \text{vârf convex}, \quad \Delta < 0 \Rightarrow \text{vârf concav}.$$

Calculăm:

| Vârf | $\Delta(V_{i-1}, V_i, V_{i+1})$ | Tip    |
|------|---------------------------------|--------|
| $A$  | 3                               | convex |
| $B$  | 2                               | convex |
| $C$  | 1                               | convex |
| $D$  | 1                               | convex |
| $E$  | -2                              | concav |
| $F$  | 3                               | convex |

Deci poligonul are atât vârfuri convexe, cât și un vârf concav.

### De ce sunt principale?

Un vârf este principal dacă segmentul care unește cei doi vecini ai săi nu intersectează laturi neadiacente ale poligonului. Pentru exemplul ales, segmentele relevante sunt:

$$FB, AC, BD, CE, DF, EA.$$

Se verifică direct din desen sau prin teste de intersecție de segmente că niciunul dintre aceste segmente nu taie o latură neadiacentă a poligonului. Prin urmare, toate cele 6 vârfuri sunt principale.

## 5 Exercițiul 5 - Număr de triunghiuri și muchii într-o triangulare

**Enunț.** Fie:

$$M = \{A_i \mid i = 0, \dots, 50\} \cup \{B_i \mid i = 0, \dots, 40\} \cup \{C_i \mid i = 0, \dots, 30\},$$

unde:

$$A_i = (i + 10, 0), \quad B_i = (0, i + 30), \quad C_i = (-i, -i).$$

Trebuie determinat numărul de triunghiuri și numărul de muchii ale unei triangulări a lui  $M$ .

### Pasul 1: numărul total de puncte

Avem:

$$|\{A_i \mid i = 0, \dots, 50\}| = 51,$$

$$|\{B_i \mid i = 0, \dots, 40\}| = 41,$$

$$|\{C_i \mid i = 0, \dots, 30\}| = 31.$$

Punctele sunt distincte între ele, deci:

$$n = 51 + 41 + 31 = 123.$$

### Pasul 2: numărul de puncte de pe frontiera acoperirii convexe

Cele trei familii de puncte sunt pe trei direcții:

- $A_i$  sunt pe axa  $Ox$ , de la  $(10, 0)$  la  $(60, 0)$ ;
- $B_i$  sunt pe axa  $Oy$ , de la  $(0, 30)$  la  $(0, 70)$ ;
- $C_i$  sunt pe dreapta  $y = x$ , de la  $(0, 0)$  la  $(-30, -30)$ .

Frontiera acoperirii convexe este determinată de cele trei puncte extreme:

$$A_{50} = (60, 0), \quad B_{40} = (0, 70), \quad C_{30} = (-30, -30).$$

Prin urmare:

$$k = 3.$$

### Pasul 3: aplicarea formulelor

Numărul de triunghiuri este:

$$n_t = 2n - k - 2.$$

Înlocuim:

$$n_t = 2 \cdot 123 - 3 - 2 = 246 - 5 = 241.$$

Deci:

$$\boxed{n_t = 241}.$$

Numărul de muchii este:

$$n_m = 3n - k - 3.$$

Înlocuim:

$$n_m = 3 \cdot 123 - 3 - 3 = 369 - 6 = 363.$$

Deci:

$$\boxed{n_m = 363}.$$

## Observație

Dacă într-un material apare rezultatul 343 pentru numărul de muchii, acela este o eroare de calcul aritmetic, deoarece:

$$3 \cdot 123 - 3 - 3 = 363,$$

nu 343.

## 6 Exercițiul 6 - Exemplu de mulțime cu 6 triunghiuri și 11 muchii

**Cerință.** Să se dea un exemplu de mulțime din  $\mathbb{R}^2$  care admite o triangulare cu 6 triunghiuri și 11 muchii.

### Folosim formulele

Pentru o mulțime de  $n$  puncte, dintre care  $k$  sunt pe frontiera acoperirii convexe:

$$n_t = 2n - k - 2, \quad n_m = 3n - k - 3.$$

Dorim:

$$n_t = 6, \quad n_m = 11.$$

Deci avem sistemul:

$$\begin{cases} 2n - k - 2 = 6, \\ 3n - k - 3 = 11. \end{cases}$$

Echivalent:

$$\begin{cases} 2n - k = 8, \\ 3n - k = 14. \end{cases}$$

Scădem prima ecuație din a doua:

$$n = 6.$$

Înlocuim în prima:

$$2 \cdot 6 - k = 8 \Rightarrow 12 - k = 8 \Rightarrow k = 4.$$

Așadar, avem nevoie de o mulțime cu:

$$n = 6 \text{ puncte totale}$$

și:

$$k = 4 \text{ puncte pe frontiera acoperirii convexe}.$$

### Exemplu concret

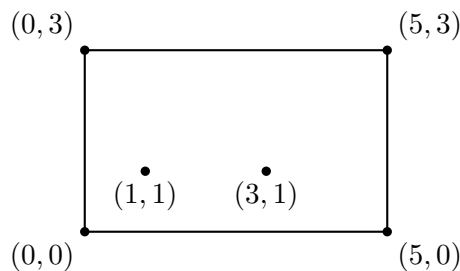
Luăm cele 4 colțuri ale unui dreptunghi și două puncte interioare:

$$M = \{(0, 0), (5, 0), (5, 3), (0, 3), (1, 1), (3, 1)\}.$$

Aici, punctele de pe frontiera acoperirii convexe sunt:

$$(0, 0), (5, 0), (5, 3), (0, 3),$$

deci  $k = 4$ , iar celelalte două sunt interioare.



Verificăm:

$$n_t = 2 \cdot 6 - 4 - 2 = 12 - 6 = 6,$$

$$n_m = 3 \cdot 6 - 4 - 3 = 18 - 7 = 11.$$

Deci mulțimea dată satisface cerința.



## 7 Exercițiul 7 - Numărul de muchii în funcție de $\alpha$

**Enunț.** În  $\mathbb{R}^2$  fie punctele:

$$P_1 = (1, 7), \quad P_2 = (5, 7), \quad P_3 = (7, 5), \quad P_4 = (1, 3), \quad P_5 = (5, 3),$$

și:

$$P_6 = (\alpha - 1, 5), \quad \alpha \in \mathbb{R}.$$

Trebuie discutat, în funcție de  $\alpha$ , numărul de muchii ale unei triangulări asociate mulțimii.

### Formula folosită

Pentru  $n$  puncte, dintre care  $k$  sunt pe frontiera acoperirii convexe, numărul de muchii este:

$$n_m = 3n - k - 3.$$

Punctele  $P_1, P_2, P_3, P_4, P_5$  determină un pentagon convex. Punctul  $P_6$  se deplasează pe dreapta:

$$y = 5,$$

având abscisa:

$$x = \alpha - 1.$$

### Cazul 1: $\alpha \leq 2$

Dacă:

$$\alpha - 1 \leq 1 \iff \alpha \leq 2,$$

atunci  $P_6$  este în exteriorul sau pe frontiera pentagonului inițial. Avem:

$$n = 6, \quad k = 6.$$

Prin urmare:

$$n_m = 3 \cdot 6 - 6 - 3 = 18 - 9 = 9.$$

Deci:

$$\boxed{n_m = 9, \quad \alpha \in (-\infty, 2]}.$$

### Cazul 2: $2 < \alpha < 8$

Dacă:

$$1 < \alpha - 1 < 7 \iff 2 < \alpha < 8,$$

atunci  $P_6$  se află în interiorul pentagonului. Avem:

$$n = 6, \quad k = 5.$$

Prin urmare:

$$n_m = 3 \cdot 6 - 5 - 3 = 18 - 8 = 10.$$

Deci:

$$\boxed{n_m = 10, \quad \alpha \in (2, 8)}.$$

**Cazul 3:**  $\alpha = 8$ 

Dacă:

$$\alpha = 8,$$

atunci:

$$P_6 = (8 - 1, 5) = (7, 5) = P_3.$$

Deci  $P_6$  coincide cu  $P_3$ , iar mulțimea are efectiv doar 5 puncte distincte:

$$n = 5, \quad k = 5.$$

Prin urmare:

$$n_m = 3 \cdot 5 - 5 - 3 = 15 - 8 = 7.$$

Deci:

$$\boxed{n_m = 7, \quad \alpha = 8}.$$

**Cazul 4:**  $\alpha > 8$ 

Dacă:

$$\alpha - 1 > 7 \iff \alpha > 8,$$

atunci  $P_6$  este din nou în exteriorul pentagonului. Avem:

$$n = 6, \quad k = 6.$$

Prin urmare:

$$n_m = 3 \cdot 6 - 6 - 3 = 9.$$

Deci:

$$\boxed{n_m = 9, \quad \alpha \in (8, \infty)}.$$

**Răspuns final**

$$\boxed{n_m(\alpha) = \begin{cases} 9, & \alpha \in (-\infty, 2], \\ 10, & \alpha \in (2, 8), \\ 7, & \alpha = 8, \\ 9, & \alpha \in (8, \infty). \end{cases}}$$

## 8 Exercițiul 8 - Inegalități pentru grafuri planare

**Enunț.** Fie  $G$  un graf planar conex, cu:

$$v = \text{numărul de noduri}, \quad m = \text{numărul de muchii}, \quad f = \text{numărul de fețe}.$$

Se presupune că fiecare vârf are gradul cel puțin 3. Trebuie demonstrate inegalitățile:

$$\begin{aligned} v &\leq \frac{2}{3}m, & m &\leq 3v - 6, \\ m &\leq 3f - 6, & f &\leq \frac{2}{3}m, \\ v &\leq 2f - 4, & f &\leq 2v - 4. \end{aligned}$$

### Formula lui Euler

Pentru un graf planar conex:

$$v - m + f = 2.$$

Echivalent:

$$f = 2 - v + m,$$

respectiv:

$$v = 2 + m - f.$$

**Demonstrația primei inegalități:**  $v \leq \frac{2}{3}m$

Fiecare vârf are grad cel puțin 3, deci:

$$\sum_{x \in V} \deg(x) \geq 3v.$$

Dar suma gradelor este:

$$\sum_{x \in V} \deg(x) = 2m.$$

Prin urmare:

$$2m \geq 3v.$$

Împărțim la 3:

$$\boxed{v \leq \frac{2}{3}m}.$$

**Demonstrația inegalității:**  $f \leq \frac{2}{3}m$

Într-un graf planar simplu, fiecare față este mărginită de cel puțin 3 muchii. Suma gradelor fețelor este  $2m$ , deoarece fiecare muchie este incidentă cu două fețe. Deci:

$$2m \geq 3f.$$

Rezultă:

$$\boxed{f \leq \frac{2}{3}m}.$$

**Demonstrația inegalității:  $m \leq 3v - 6$** 

Din formula lui Euler:

$$f = 2 - v + m.$$

Folosim  $2m \geq 3f$ :

$$2m \geq 3(2 - v + m).$$

Dezvoltăm:

$$2m \geq 6 - 3v + 3m.$$

Mutăm termenii:

$$-m \geq 6 - 3v.$$

Înmulțim cu  $-1$  și schimbăm sensul inegalității:

$$\boxed{m \leq 3v - 6}.$$

**Demonstrația inegalității:  $m \leq 3f - 6$** 

Din formula lui Euler:

$$v = 2 + m - f.$$

Folosim  $2m \geq 3v$ :

$$2m \geq 3(2 + m - f).$$

Dezvoltăm:

$$2m \geq 6 + 3m - 3f.$$

Mutăm termenii:

$$-m \geq 6 - 3f.$$

Înmulțim cu  $-1$ :

$$\boxed{m \leq 3f - 6}.$$

**Demonstrația inegalității:  $v \leq 2f - 4$** 

Din formula lui Euler:

$$v = 2 + m - f.$$

Folosim inegalitatea demonstrată anterior:

$$m \leq 3f - 6.$$

Atunci:

$$v = 2 + m - f \leq 2 + (3f - 6) - f.$$

Deci:

$$v \leq 2f - 4.$$

Prin urmare:

$$\boxed{v \leq 2f - 4}.$$

### Demonstrația inegalității: $f \leq 2v - 4$

Din formula lui Euler:

$$f = 2 - v + m.$$

Folosim:

$$m \leq 3v - 6.$$

Atunci:

$$f = 2 - v + m \leq 2 - v + (3v - 6).$$

Deci:

$$f \leq 2v - 4.$$

Prin urmare:

$$\boxed{f \leq 2v - 4}.$$

### Exemplu unde au loc egalități

Un exemplu este graful complet  $K_4$ , desenat planar ca graful tetraedrului. Pentru  $K_4$  avem:

$$v = 4, \quad m = 6, \quad f = 4.$$

Verificăm egalitățile:

$$v = 4 = \frac{2}{3} \cdot 6 = \frac{2}{3}m,$$

$$m = 6 = 3 \cdot 4 - 6 = 3v - 6,$$

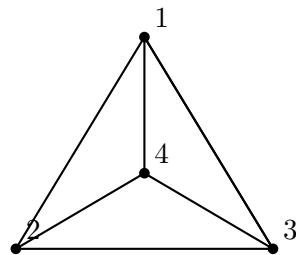
$$m = 6 = 3 \cdot 4 - 6 = 3f - 6,$$

$$f = 4 = \frac{2}{3} \cdot 6 = \frac{2}{3}m,$$

$$v = 4 = 2 \cdot 4 - 4 = 2f - 4,$$

$$f = 4 = 2 \cdot 4 - 4 = 2v - 4.$$

Deci în  $K_4$  au loc egalități în toate relațiile cerute.



## Rezumat rapid pentru examen

- **Jarvis' march:** alegi un pivot și îl înlocuiești când găsești un punct mai extrem. Pentru dreapta lui  $\overrightarrow{AB}$  folosești  $\Delta(A, B, C) < 0$ .

- **Orientare:**

$$\Delta(A, B, C) = (x_B - x_A)(y_C - y_A) - (y_B - y_A)(x_C - x_A).$$

- **Galeria de Artă:** pentru  $n$  vârfuri sunt suficiente  $\lfloor n/3 \rfloor$  camere. Metoda: triangulare + 3-colorare + alegerea culorii minime.
- **Triangulare poligon:** un poligon cu  $n$  vârfuri are  $n - 3$  diagonale și  $n - 2$  triunghiuri.
- **Triangulare mulțime de puncte:**

$$n_t = 2n - k - 2, \quad n_m = 3n - k - 3.$$

- **Euler pentru graf planar conex:**

$$v - m + f = 2.$$

- **Sume de grade:**

$$\sum_{x \in V} \deg(x) = 2m, \quad \sum_F \deg(F) = 2m.$$

- Dacă fiecare vârf are grad cel puțin 3, atunci  $2m \geq 3v$ .
- Dacă fiecare față are cel puțin 3 muchii, atunci  $2m \geq 3f$ .

**Anexa E - Seminarul 7: Voronoi, dualitate, semiplane, Delaunay**

Rezolvare detaliata – Seminar 7  
Algoritmi Avansati: Voronoi, dualitate, semiplane, Delaunay

## Contents

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| <b>Idei esentiale pentru examen</b>                                             | <b>2</b>  |
| <b>1 Exercițiul 1 – Numar de semidrepte in diagrama Voronoi</b>                 | <b>3</b>  |
| <b>2 Exercițiul 2 – Inegalitati pentru diagrama Voronoi</b>                     | <b>5</b>  |
| 2.1 Punctul a) . . . . .                                                        | 5         |
| 2.2 Punctul b) . . . . .                                                        | 6         |
| <b>3 Exercițiul 3 – Numarul de semidrepte in functie de <math>\alpha</math></b> | <b>8</b>  |
| <b>4 Exercițiul 4 – Dualitate punct-dreapta</b>                                 | <b>10</b> |
| 4.1 Punctul (i) . . . . .                                                       | 10        |
| 4.2 Punctul (ii) . . . . .                                                      | 11        |
| <b>5 Exercițiul 5 – Intersectii de semiplane</b>                                | <b>13</b> |
| 5.1 Punctul a) . . . . .                                                        | 13        |
| 5.2 Punctul b) . . . . .                                                        | 14        |
| <b>6 Exercițiul 6 – Semiplane din normale exterioare</b>                        | <b>16</b> |
| <b>7 Exercițiul 7 suplimentar – MST este subgraf al triangulatiei Delaunay</b>  | <b>17</b> |
| <b>Rezumat final pentru examen</b>                                              | <b>19</b> |



## Idei esentiale pentru examen

- Pentru o multime de situri din plan, muchiile nemarginite ale diagramei Voronoi sunt semidrepte. Numarul lor este egal cu numarul de puncte aflate pe frontiera acoperirii convexe.
- Pentru  $n$  situri necoliniare, daca  $n_v$  este numarul de varfuri Voronoi si  $n_m$  numarul de muchii Voronoi, atunci:

$$n_v \leq 2n - 5, \quad n_m \leq 3n - 6.$$

- Dualitatea punct-dreapta folosita in curs:

$$p = (p_x, p_y) \iff p^* : y = p_x x - p_y,$$

iar pentru o dreapta neverticala

$$d : y = mx + n \iff d^* = (m, -n).$$

Incidenta se pastreaza invers: daca punctul  $p$  se afla pe dreapta  $d$ , atunci punctul dual  $d^*$  se afla pe dreapta duala  $p^*$ .

- Un semiplan asociat unei normale exterioare  $(a, b, c)$  are forma:

$$ax + by + c \leq 0.$$

- Arborele partial de cost minim este subgraf al triangulatiei Delaunay:

$$\text{MST}(P) \subseteq \text{DT}(P).$$

# 1 Exercițiul 1 – Numar de semidrepte in diagrama Voronoi

## Enunt

Sa se dea un exemplu de multime

$$M = \{A_1, A_2, A_3, A_4, A_5, A_6\} \subset \mathbb{R}^2$$

astfel incat diagrama Voronoi asociata lui  $M$  sa contina exact patru semidrepte, iar diagrama Voronoi asociata lui  $M \setminus \{A_1\}$  sa contina exact cinci semidrepte.

## Observatie teoretica

Pentru o diagrama Voronoi in plan:

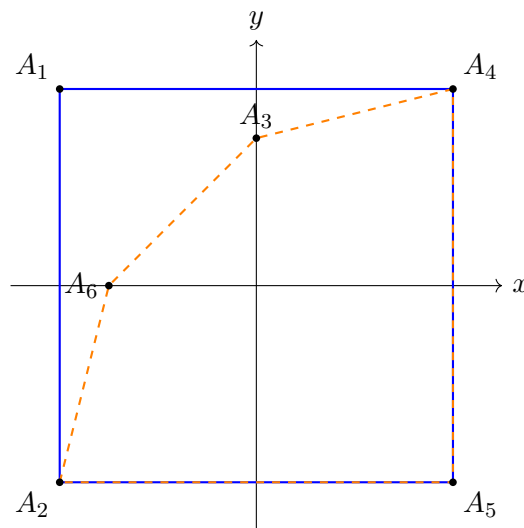
$$\boxed{\text{numarul de semidrepte Voronoi} = \text{numarul de puncte de pe } \text{CH}(M)}.$$

Intuitiv, fiecare punct extrem de pe acoperirea convexa are o celula Voronoi nemarginita. Muchiile nemarginite ale acestor celule apar ca semidrepte in diagrama.

## Constructia multimii

Alegem punctele:

$$\begin{aligned} A_1 &= (-4, 4), & A_2 &= (-4, -4), & A_3 &= (0, 3), \\ A_4 &= (4, 4), & A_5 &= (4, -4), & A_6 &= (-3, 0). \end{aligned}$$



## Justificare

Pentru multimea  $M$ , punctele de pe frontiera acoperirii convexe sunt:

$$A_1, A_2, A_5, A_4.$$

Punctele  $A_3$  si  $A_6$  sunt in interiorul patrulaterului determinat de acestea. Prin urmare:

$$|\text{CH}(M)| = 4.$$

Deci diagrama Voronoi asociata lui  $M$  are exact:

$$\boxed{4 \text{ semidrepte}}.$$

Pentru multimea  $M \setminus \{A_1\}$  raman punctele:

$$A_2, A_3, A_4, A_5, A_6.$$

Cu alegerea de mai sus, toate cele cinci puncte sunt pe frontiera acoperirii convexe. Deci:

$$|\text{CH}(M \setminus \{A_1\})| = 5.$$

Prin urmare diagrama Voronoi asociata lui  $M \setminus \{A_1\}$  are exact:

$$\boxed{5 \text{ semidrepte}}.$$

### **Concluzie pentru examen**

Aici nu trebuie desenata efectiv diagrama Voronoi. Este suficient sa folosim legatura:

$$\text{semidrepte Voronoi} \iff \text{puncte pe acoperirea convexa}.$$

## 2 Exercițiul 2 – Inegalități pentru diagrama Voronoi

### 2.1 Punctul a)

Fie o mulțime cu  $n$  situri necoliniare. Pentru diagrama Voronoi asociată trebuie demonstrate inegalitățile:

$$\boxed{n_v \leq 2n - 5}, \quad \boxed{n_m \leq 3n - 6},$$

unde:

- $n_v$  = numărul de varfuri Voronoi;
- $n_m$  = numărul de muchii Voronoi;
- $n$  = numărul de situri.

#### Pasul 1: transformăm diagrama într-un graf planar finit

Diagrama Voronoi poate avea semidrepte, deci muchii nemarginite. Pentru a aplica formula lui Euler, introducem un varf suplimentar  $v_\infty$ , la infinit, prin care trec toate semidreptele.

Astfel obținem un graf planar conex  $G$  cu:

$$V_G = n_v + 1,$$

$$E_G = n_m,$$

$$F_G = n.$$

Fetele sunt în corespondență cu siturile inițiale.

#### Pasul 2: folosim formula lui Euler

Pentru un graf planar conex:

$$V - E + F = 2.$$

În cazul nostru:

$$(n_v + 1) - n_m + n = 2.$$

Rezultă:

$$\boxed{n_m = n_v + n - 1}. \quad (1)$$

#### Pasul 3: numărăm incidentele varf-muchie

Fiecare muchie are două capete, deci numărul total de incidente este:

$$2n_m.$$

Fiecare varf Voronoi are grad cel puțin 3, iar  $v_\infty$  are și el grad cel puțin 3, deoarece punctele sunt necoliniare. Deci:

$$2n_m \geq 3(n_v + 1). \quad (2)$$

#### Pasul 4: deducem prima inegalitate

Inlocuim (1) in (2):

$$2(n_v + n - 1) \geq 3(n_v + 1).$$

Desfacem:

$$2n_v + 2n - 2 \geq 3n_v + 3.$$

Mutam termenii:

$$2n - 5 \geq n_v.$$

Deci:

$$\boxed{n_v \leq 2n - 5}.$$

#### Pasul 5: deducem a doua inegalitate

Din relatia:

$$n_m = n_v + n - 1$$

si din:

$$n_v \leq 2n - 5,$$

obtinem:

$$n_m \leq (2n - 5) + n - 1 = 3n - 6.$$

Prin urmare:

$$\boxed{n_m \leq 3n - 6}.$$

## 2.2 Punctul b)

Avem  $n = 5$  puncte si diagrama Voronoi are exact 5 semidrepte.

Cum numarul de semidrepte este egal cu numarul de puncte de pe frontiera acoperirii convexe, rezulta ca toate cele 5 puncte sunt pe frontiera acoperirii convexe.

Valoarea maxima teoretica ar fi:

$$n_v \leq 2n - 5 = 2 \cdot 5 - 5 = 5.$$

Dar aceasta valoare maxima se atinge numai cand fiecare varf din graful extins are grad 3. In particular,  $v_\infty$  trebuie sa aiba grad 3, adica diagrama ar trebui sa aiba 3 semidrepte. Noi avem 5 semidrepte. Deci maximul 5 nu poate fi atins.

#### Analiza cazurilor

Pentru 5 puncte toate pe acoperirea convexa, sunt posibile urmatoarele cazuri:

**Cazul 1.** Toate cele 5 puncte sunt conciclice. Atunci exista un singur punct egal departat de toate situarile, deci:

$$\boxed{n_v = 1}.$$

**Cazul 2.** Exact 4 puncte sunt conciclice, iar al cincilea nu este pe acelasi cerc. Atunci cele 4 puncte conciclice produc un varf Voronoi de grad mai mare, iar restul structurii mai adauga un varf. Deci:

$$\boxed{n_v = 2}.$$

**Cazul 3.** Nu exista 4 puncte conciclice. Atunci diagrama este in pozitie generala fata de cocircularitate, dar pentru ca toate cele 5 puncte sunt pe frontiera convexa, numarul de varfuri este:

$$\boxed{n_v = 3}.$$

### Concluzie

Numarul de varfuri posibil este:

$$\boxed{n_v \in \{1, 2, 3\}}.$$

Maximul teoretic  $2n - 5 = 5$  nu este atins, deoarece diagrama are 5 semidrepte, nu 3.

### 3 Exercițiul 3 – Numarul de semidrepte in functie de $\alpha$

#### Enunt

Fie punctele:

$$O = (0, 0), \quad A = (\alpha, 0), \quad B = (1, 1), \quad C = (2, 0), \quad D = (1, -1).$$

Trebuie discutat, in functie de  $\alpha$ , numarul de muchii de tip semidreapta ale diagramei Voronoi asociate multimii:

$$\{O, A, B, C, D\}.$$

#### Observatia principala

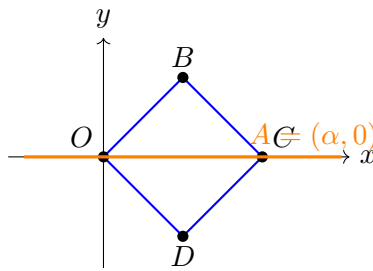
Din nou folosim faptul ca:

|                                                                             |
|-----------------------------------------------------------------------------|
| numarul de semidrepte Voronoi = numarul de puncte de pe acoperirea convexa. |
|-----------------------------------------------------------------------------|

Punctele  $O, B, C, D$  formeaza un patrat rotit:

$$O = (0, 0), \quad B = (1, 1), \quad C = (2, 0), \quad D = (1, -1).$$

Punctul  $A = (\alpha, 0)$  se misca pe axa  $Ox$ .



#### Cazuri

**Cazul 1.**  $\alpha < 0$ .

Punctul  $A$  este in exteriorul patratului, la stanga lui  $O$ . Punctele extreme sunt:

$$A, B, C, D.$$

Punctul  $O$  devine interior sau se afla pe diagonala interioara. Deci avem:

|                |
|----------------|
| 4 semidrepte . |
|----------------|

**Cazul 2.**  $\alpha = 0$ .

Atunci  $A = O$ . Multimea de situri distincte este data de punctele  $O, B, C, D$ , toate pe frontiera. Deci:

|                |
|----------------|
| 4 semidrepte . |
|----------------|

**Cazul 3.**  $0 < \alpha < 2$ .

Punctul  $A$  se afla pe segmentul  $OC$ , deci este in interiorul patratului  $OBCD$ . Punctele de pe frontiera raman:

$$O, B, C, D.$$

Deci:

|                |
|----------------|
| 4 semidrepte . |
|----------------|

**Cazul 4.**  $\alpha = 2$ .

Atunci  $A = C$ . Punctele distincte de pe frontiera sunt:

$$O, B, C, D.$$

Deci:

$$\boxed{4 \text{ semidrepte}}.$$

**Cazul 5.**  $\alpha > 2$ .

Punctul  $A$  este in exteriorul patraturii, la dreapta lui  $C$ . Punctele extreme sunt:

$$O, B, A, D.$$

Punctul  $C$  nu mai este extrem. Deci:

$$\boxed{4 \text{ semidrepte}}.$$

### Concluzie

Indiferent de valoarea lui  $\alpha$ , numarul de semidrepte este:

$$\boxed{4}.$$



## 4 Exercițiul 4 – Dualitate punct-dreapta

### 4.1 Punctul (i)

Fie punctul:

$$A = (1, 2).$$

Alegem doua drepte distincte care trec prin  $A$ :

$$d : y = 2x,$$

$$g : y = -x + 3.$$

Verificare:

$$A = (1, 2) \in d, \quad 2 = 2 \cdot 1,$$

$$A = (1, 2) \in g, \quad 2 = -1 + 3.$$

### Formula de dualitate

Pentru punctul  $p = (p_x, p_y)$ :

$$p^* : y = p_x x - p_y.$$

Pentru dreapta  $d : y = mx + n$ :

$$d^* = (m, -n).$$

### Calculam dualele

Pentru  $A = (1, 2)$ :

$$A^* : y = x - 2.$$

Pentru  $d : y = 2x + 0$ :

$$d^* = (2, 0).$$

Pentru  $g : y = -x + 3$ :

$$g^* = (-1, -3).$$

### Verificarea incidentei

Trebuie sa verificam ca punctele  $d^*$  si  $g^*$  se afla pe dreapta  $A^*$ .

Pentru  $d^* = (2, 0)$ :

$$y = x - 2 \implies 0 = 2 - 2 = 0.$$

Deci  $d^* \in A^*$ .

Pentru  $g^* = (-1, -3)$ :

$$y = x - 2 \implies -3 = -1 - 2 = -3.$$

Deci  $g^* \in A^*$ .

Prin urmare dreapta determinata de punctele  $d^*$  si  $g^*$  este chiar:

$$A^* : y = x - 2.$$

## 4.2 Punctul (ii)

Configuratia initiala este:

- patru drepte concurente intr-un punct  $M$ ;
- alegem doua dintre aceste drepte;
- pe fiecare alegem cate un punct diferit de  $M$ ;
- unim cele doua puncte printr-o dreapta.

### Ce se intampla prin dualitate?

Reguli de baza:

punct  $\longleftrightarrow$  dreapta,

dreapta  $\longleftrightarrow$  punct,

drepte concurente intr-un punct  $\longleftrightarrow$  puncte coliniare pe o dreapta.

Deci cele patru drepte care trec prin  $M$  devin patru puncte coliniare pe dreapta  $M^*$ .

Daca doua puncte  $P, Q$  sunt alese pe doua dintre dreptele initiale, atunci in dual:

- punctele  $P, Q$  devin dreptele  $P^*, Q^*$ ;
- dreapta  $PQ$  devine punctul  $(PQ)^*$ ;
- faptul ca  $P, Q \in PQ$  devine faptul ca  $(PQ)^* \in P^*$  si  $(PQ)^* \in Q^*$ .

### Configuratia duala

Configuratia duala este:

- patru puncte coliniare pe o dreapta  $d$ ;
- prin doua dintre ele trece cate o dreapta distincta de  $d$ ;
- cele doua drepte se intersecteaza intr-un punct.

### Completare la configuratie autoduala

Pentru a obtine o configuratie autoduala, o completam astfel:

- avem patru drepte concurente intr-un punct  $M$ ;
- pe fiecare dintre cele patru drepte alegem cate un punct diferit de  $M$ ;
- alegem aceste patru puncte astfel incat sa fie coliniare;
- adaugam dreapta care le contine.

Atunci configuratia contine:

- un punct prin care trec patru drepte;

- o dreapta pe care se afla patru puncte;
- incidentele se pastreaza simetric prin dualitate.

Aceasta este autoduala deoarece duala are acelasi tip de obiecte si aceleasi relatii de incidenta.

## 5 Exercițiul 5 – Intersecții de semiplane

### 5.1 Punctul a)

Sunt date semiplanele:

$$H : x + y - 3 \leq 0,$$

$$H' : -2x + y + 1 \leq 0.$$

Trebuie să dam un semiplan  $H''$  astfel încât:

$$H \cap H' \cap H''$$

să fie un triunghi dreptunghic.

Alegem:

$$\boxed{H'' : x - y - 5 \leq 0}.$$

Dreptele suport sunt:

$$L : x + y - 3 = 0,$$

$$L' : -2x + y + 1 = 0,$$

$$L'' : x - y - 5 = 0.$$

Observăm că  $L$  și  $L''$  sunt perpendiculare. Într-adevăr:

$$L : y = -x + 3 \quad \Rightarrow \quad m_L = -1,$$

$$L'' : y = x - 5 \quad \Rightarrow \quad m_{L''} = 1.$$

Produsul pantelor este:

$$m_L \cdot m_{L''} = (-1) \cdot 1 = -1.$$

Deci unghiul dintre ele este drept.

### Calculul varfurilor triunghiului

Intersecția lui  $L$  cu  $L'$ :

$$\begin{cases} x + y = 3, \\ -2x + y + 1 = 0 \end{cases} \iff \begin{cases} y = 3 - x, \\ y = 2x - 1 \end{cases}$$

$$3 - x = 2x - 1 \implies 3x = 4 \implies x = \frac{4}{3},$$

$$y = 3 - \frac{4}{3} = \frac{5}{3}.$$

Deci:

$$V_1 = \left( \frac{4}{3}, \frac{5}{3} \right).$$

Intersecția lui  $L$  cu  $L''$ :

$$\begin{cases} x + y = 3, \\ x - y = 5 \end{cases} \implies 2x = 8 \implies x = 4,$$

$$y = -1.$$

Deci:

$$V_2 = (4, -1).$$

Intersectia lui  $L'$  cu  $L''$ :

$$\begin{cases} y = 2x - 1, \\ y = x - 5 \end{cases} \implies 2x - 1 = x - 5 \implies x = -4,$$

$$y = -9.$$

Deci:

$$V_3 = (-4, -9).$$

Prin urmare intersectia celor trei semiplane este triunghiul cu varfurile:

$$\left( \frac{4}{3}, \frac{5}{3} \right), (4, -1), (-4, -9).$$

Cum laturile corespunzatoare dreptelor  $L$  si  $L''$  sunt perpendiculare, triunghiul este dreptunghic.

## 5.2 Punctul b)

Avem semiplanele:

$$H_1 : -y + 1 \leq 0 \iff y \geq 1,$$

$$H_2 : y - 5 \leq 0 \iff y \leq 5,$$

$$H_3 : -x \leq 0 \iff x \geq 0,$$

$$H_4 : x - y + a \leq 0 \iff x \leq y - a.$$

Primele trei semiplane dau banda nemarginita:

$$1 \leq y \leq 5, \quad x \geq 0.$$

Al patrulea semiplan impune:

$$0 \leq x \leq y - a.$$

Ca intersectia sa fie nevida, trebuie sa existe  $y \in [1, 5]$  astfel incat:

$$y - a \geq 0 \iff y \geq a.$$

Cum cel mai mare  $y$  permis este 5, rezulta ca intersectia este nevida numai daca:

$$a \leq 5.$$

### Cazuri

**Cazul 1.**  $a > 5$ .

Nu exista  $y \in [1, 5]$  cu  $y \geq a$ . Deci intersectia este vida:

$$\emptyset.$$

**Cazul 2.**  $a = 5$ .

Singura valoare posibila este  $y = 5$ , iar atunci:

$$0 \leq x \leq y - a = 5 - 5 = 0.$$

Deci  $x = 0$  si intersectia este un singur punct:

$$(0, 5).$$

**Cazul 3.**  $1 \leq a < 5$ .

Intersectia este un triunghi. Varfurile sunt:

$$(0, 5), \quad (0, a), \quad (5 - a, 5).$$

Pentru  $a = 1$ , punctul  $(0, a)$  devine  $(0, 1)$ .

Deci:

triunghi

.

**Cazul 4.**  $a < 1$ .

Dreapta  $x = y - a$  intersecteaza atat dreapta  $y = 1$ , cat si dreapta  $y = 5$  la abscise pozitive:

$$y = 1 \Rightarrow x = 1 - a > 0,$$

$$y = 5 \Rightarrow x = 5 - a > 0.$$

Intersectia este un trapez cu varfurile:

$$(0, 1), \quad (0, 5), \quad (5 - a, 5), \quad (1 - a, 1).$$

Deci:

trapez

.

**Rezumat**

$$H_1 \cap H_2 \cap H_3 \cap H_4 = \begin{cases} \emptyset, & a > 5, \\ \{(0, 5)\}, & a = 5, \\ \text{triunghi}, & 1 \leq a < 5, \\ \text{trapez}, & a < 1. \end{cases}$$

## 6 Exercițiul 6 – Semiplane din normale exterioare

### Enunț

Trebuie scrise inecuațiile semiplanelor corespunzătoare dacă normalele exterioare ale fetelor standard sunt coliniare cu vectorii:

$$(0, 1, -1), \quad (0, 1, 0), \quad (0, 0, -1), \quad (0, -1, 0), \quad (0, -1, -1).$$

### Formula folosită

Dacă normala exterioară este coliniară cu vectorul:

$$(a, b, c),$$

atunci semiplanul corespunzător este:

$$\boxed{ax + by + c \leq 0}.$$

### Aplicare pe fiecare vector

| Vector normal | Semiplan             | Forma simplificată |
|---------------|----------------------|--------------------|
| $(0, 1, -1)$  | $0x + 1y - 1 \leq 0$ | $y \leq 1$         |
| $(0, 1, 0)$   | $0x + 1y + 0 \leq 0$ | $y \leq 0$         |
| $(0, 0, -1)$  | $0x + 0y - 1 \leq 0$ | $-1 \leq 0$        |
| $(0, -1, 0)$  | $0x - y + 0 \leq 0$  | $y \geq 0$         |
| $(0, -1, -1)$ | $0x - y - 1 \leq 0$  | $y \geq -1$        |

Sistemul este:

$$\begin{cases} y \leq 1, \\ y \leq 0, \\ -1 \leq 0, \\ y \geq 0, \\ y \geq -1. \end{cases}$$

Inecuația  $-1 \leq 0$  este mereu adevărată, deci nu restricționează intersecția.

Condițiile importante sunt:

$$y \leq 0, \quad y \geq 0.$$

De aici:

$$\boxed{y = 0}.$$

Coordonata  $x$  nu apare în restricții, deci poate fi orice număr real.

### Concluzie

Intersecția semiplanelor este dreapta:

$$\boxed{\{(x, 0) \mid x \in \mathbb{R}\}}.$$

## 7 Exercițiul 7 suplimentar – MST este subgraf al triangulației Delaunay

### Enunț

Să se demonstreze că arborele parțial de cost minim al unei mulțimi de puncte  $P$  este un subgraf al triangulației Delaunay:

$$\boxed{\text{MST}(P) \subseteq \text{DT}(P)}.$$

### Ideea demonstrației

Vom folosi două fapte standard:

1. Dacă o muchie  $pq$  aparține unui MST, atunci discul cu diametrul  $pq$  nu conține niciun alt punct din  $P$  în interior.
2. Dacă există un cerc gol care trece prin  $p$  și  $q$ , atunci  $pq$  este muchie Delaunay.

### Pasul 1: luăm o muchie oarecare din MST

Fie:

$$pq \in \text{MST}(P).$$

Vom arăta că:

$$pq \in \text{DT}(P).$$

Considerăm discul  $D_{pq}$  care are segmentul  $pq$  ca diametru.

### Pasul 2: presupunem contrariul

Presupunem că există un punct:

$$r \in P \setminus \{p, q\}$$

în interiorul discului  $D_{pq}$ .

Din geometria cercului cu diametrul  $pq$ , dacă  $r$  este în interior, atunci unghiul  $\angle prq$  este mai mare de  $90^\circ$ , iar în particular rezultă:

$$|pr| < |pq|,$$

$$|qr| < |pq|.$$

### Pasul 3: folosim proprietatea de MST

Stergem muchia  $pq$  din arborele parțial de cost minim. Arborele se rupe în două componente conexe:

$$C_p \quad \text{și} \quad C_q,$$

unde:

$$p \in C_p, \quad q \in C_q.$$

Punctul  $r$  se află în una dintre cele două componente.



- Daca  $r \in C_p$ , atunci muchia  $rq$  conecteaza cele doua componente  $C_p$  si  $C_q$ . Dar:

$$|rq| < |pq|.$$

Inlocuind  $pq$  cu  $rq$ , obtinem un arbore de cost mai mic, contradictie cu minimalitatea MST.

- Daca  $r \in C_q$ , atunci muchia  $rp$  conecteaza cele doua componente  $C_q$  si  $C_p$ . Dar:

$$|rp| < |pq|.$$

Inlocuind  $pq$  cu  $rp$ , obtinem din nou un arbore de cost mai mic, contradictie.

Deci presupunerea a fost falsa. Prin urmare discul cu diametrul  $pq$  este gol:

$$D_{pq}^\circ \cap P = \emptyset.$$

#### **Pasul 4: concluzia Delaunay**

Cercul cu diametrul  $pq$  trece prin  $p$  si  $q$  si nu contine niciun alt punct din  $P$  in interior. Prin criteriul cercului gol, muchia  $pq$  este o muchie Delaunay.

Am aratat ca orice muchie  $pq$  din MST este si muchie Delaunay. Deci:

$$\boxed{\text{MST}(P) \subseteq \text{DT}(P)}.$$

#### **Observatie pentru examen**

Daca exista degenerari, de exemplu mai multe puncte conciclice, afirmatia se interpreteaza pentru graful Delaunay sau pentru o triangulatie Delaunay aleasa astfel incat sa contina muchiile respective. In pozitie generala, demonstratia de mai sus este suficienta.

## Rezumat final pentru examen

| Concept                  | Ce trebuie retinut                                                                                                                        |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Semidrepte Voronoi       | Numarul lor este egal cu numarul de puncte de pe frontiera acoperirii convexe.                                                            |
| Inegalitati Voronoi      | Pentru $n$ situri necoliniare: $n_v \leq 2n - 5$ , $n_m \leq 3n - 6$ . Se demonstreaza cu Euler si numarare de incidente.                 |
| Dualitate                | $p = (p_x, p_y) \mapsto p^* : y = p_x x - p_y$ , iar $d : y = mx + n \mapsto d^* = (m, -n)$ . Incidenta se transforma in incidenta duala. |
| Semiplane                | Din normala $(a, b, c)$ se obtine semiplanul $ax + by + c \leq 0$ .                                                                       |
| Intersectii de semiplane | Se transforma inegalitatile in restrictii simple: sus/jos/stanga/dreapta si se discuta cazurile parametru-lui.                            |
| MST si Delaunay          | Daca $pq \in \text{MST}$ , discul cu diametrul $pq$ este gol. Deci exista cerc gol prin $p, q$ , deci $pq \in \text{DT}$ .                |